6.7.2 Model 类

Model 类是 Gurobi 最重要的类，其中包含建模，添加决策变量和约束、求解等相关函数。Model 类的部分函数如表 6.2 所示。

表 6.2 Model 类的部分函数

函 数 名	功 能
`Model(name="", env=defaultEnv)`	建立模型对象
`Model.addConstr (lhs, sense= None, rhs=None, name="")`	在模型对象中添加一条约束
`Model.addConstrs (generator, name="")`	在模型对象中添加多条约束
`Model.addGenConstrXxx ()`	向模型对象中添加广义约束（具体函数名如下）
`addGenConstrMax`	添加约束 $y=\max\{x_1,x_2,\cdots,c\}$
`addGenConstrMin`	添加约束 $y=\min\{x_1,x_2,\cdots,c\}$
`addGenConstrAbs`	添加约束 $y=\lvert x\rvert$
`addGenConstrAnd`	添加约束 $y=x_1\wedge x_2\wedge\cdots$
`addGenConstrOr`	添加约束 $y=x_1\vee x_2\vee\cdots$
`addGenConstrIndicator`	添加约束 $y=1\rightarrow ax\leqslant b$ （an indicator constraint）
`addGenConstrPWL`	添加约束 $y=\mathrm{PWL}(x)$（一个由若干分段点定义的分段线性函数）
`addGenConstrPoly`	添加约束 $y=p_0x^d+p_1x^{d-1}+\cdots+p_{d-1}x+p_d$
`addGenConstrExp`	添加约束 $y=e^x$
`addGenConstrExpA`	添加约束 $y=a^x$
`addGenConstrLog`	添加约束 $y=\ln x$
`addGenConstrLogA`	添加约束 $y=\log_a x$
`addGenConstrPow`	添加约束 $y=x^a$
`addGenConstrSin`	添加约束 $y=\sin x$
`addGenConstrCos`	添加约束 $y=\cos x$
`addGenConstrTan`	添加约束 $y=\tan x$
`Model.addLConstr (lhs, sense= None, rhs=None, name="")`	在模型对象中添加一个线性约束（addLConstr() 比 addConstr() 快很多）
`Model.addMConstrs (A, x, sense, b, names="")`	以矩阵形式向模型中添加多个线性约束
`Model.addMQConstr (Q, c, sense, rhs, xQ_L=None, xQ_R=None, xc=None, name="" )`	以矩阵形式向模型中添加多个二次约束
`Model.addMVar (shape, lb=0.0, ub=GRB.INFINITY, obj=0.0, vtype=GRB.CONTINUOUS, name="")`	在模型中添加一个 MVar 对象
`Model.addQConstr (lhs, sense= None, rhs=None, name="")`	在模型中添加一个二次约束

续表

函 数 名	功 能
Model.addRange (expr, lower, upper, name="")	在模型中添加一个范围约束 lb ⩽ expr ⩽ ub
Model.addVar (lb=0.0, ub=GRB.INFINITY, obj=0.0, vtype=GRB.CONTINUOUS, name="", column=None)	在模型中添加一个决策变量
Model.addVars (*indices, lb=0.0, ub=GRB.INFINITY, obj=0.0, vtype=GRB.CONTINUOUS, name="")	在模型中添加多个决策变量
Model.cbCut (lhs, sense, rhs)	在 callback 函数中向 MIP 中添加一个割平面。（注意：该函数只能在 callback 函数中的 where 参数等于 GRB.Callback.MIPNODE 时才会被调用）
Model.cbGet (what)	在 user callback 函数中查询优化器相关信息
Model.cbGetNodeRel (vars)	在 callback 函数中获取当前节点的线性松弛的解中决策变量的取值
Model.cbGetSolution (vars)	在 callback 函数中获取 MIP 的当前解（整数解）
Model.cbLazy (lhs, sense, rhs)	在 callback 函数中向 MIP 模型中添加一个 lazy 约束（惰性约束）
Model.cbSetSolution (vars, solution)	将一个已知解或者已知的部分解传给当前模型
Model.cbUseSolution ( )	当使用 cbSetSolution 导入一个解之后，可以调用 cbUseSolution 来计算这个解对应的目标函数
Model.chgCoeff (constr, var, newvalue)	改变模型中一个约束中的一个变量的系数
Model.computeIIS (void)	计算一个 IIS
Model.copy ()	复制模型。注意，由于程序汇总可能使用了 lazy 更新，因此，调用该函数前，最好先调用 update 函数
Model.feasRelaxS (relaxobjtype, minrelax, vrelax, crelax)	修改模型对象以获得一个可行的松弛模型
Model.feasRelax (relaxobjtype, minrelax, vars, lbpen, ubpen, constrs, rhspen)	修改模型对象以获得一个可行的松弛模型
Model.fixed ()	创建一个固定的 MIP 模型。在该固定的模型中，整数变量被固定为其在 MIP 解中的值，并且连续变量也被固定，以满足 SOS 约束和广义约束。被固定的模型中，没有任何整数约束、SOS 约束和广义约束
Model.getA ()	查询模型的线性约束矩阵
Model.getAttr (attrname, objs=None)	查询属性的值

续表

函 数 名	功 能
Model.getCoeff (constr, var)	查询一个变量 var 在线性约束 constr 中的系数
Model.getCol (var)	提取决策变量 var 对应的列，包括目标函数系数和对应列的约束系数，返回一个 Column 对象
Model.getConstrByName (name)	根据约束名提取约束。如果多个约束有相同的名字，则该方法将会任意选择一个返回
Model.getConstrs ()	提取多个约束
Model.getGenConstrXxx (genconstr)	提取广义约束的数据（其中 Xxx 是所有广义约束的类型，调用时对应到相应的类型即可，可选类型与之前添加约束的部分的类型相同）
Model.getGenConstrs ()	提取模型中的所有广义约束
Model.getJSONSolution ()	在调用 optimize 函数后，调用该函数提取得到的解和相关的模型属性，返回成一个 JSON 字符串
Model.getObjective (index=None)	提取模型的目标函数（多个目标函数返回多个目标值）
Model.getParamInfo (paramname)	提取模型的参数信息，包括类型、当前值、最大最小允许值和默认值
Model.getPWLObj (var)	提取一个变量 var 的分段线性目标函数值
Model.getQConstrs ()	提取模型中的所有二次约束
Model.getQCRow (qconstr)	提取二次约束的左端项表达式，返回一个 QuadExpr 对象
Model.getRow (constr)	提取约束 constr 的左端项表达式，返回一个 LinExpr 对象
Model.getSOS (sos)	提取 SOS 约束 sos 的信息，返回结果为一个 tuple，包含 SOS 类型（1 或 2）、参与 SOS 的变量对象以及对应的 SOS 权重
Model.getSOSs ()	提取模型中的所有 SOS 约束
Model.getTuneResult ()	提取之前调用 tune 之后的调优结果
Model.getVarByName (name)	根据变量名提取变量
Model.getVars ()	提取模型中的所有变量
Model.message (msg)	向 Gurobi 的日志文件中添加一个字符串
Model.optimize (callback=None)	求解模型。如果是连续型模型，则调用单纯形法或者内点法进行求解；如果是 MIP 模型，则调用 branch-and-cut 算法求解
Model.presolve ()	对模型实行预求解
Model.printAttr (attrs, filter='*')	打印出一个或者多个模型的属性值
Model.printQuality ()	打印出解的质量统计信息，包括约束违反、整数违反信息等
Model.printStats ()	打印出模型的统计信息（包括约束和变量、非 0 约束系数的个数和最小与最大的约束系数等）
Model.read (filename)	从文件读入数据，导入给模型，包括模型数据和参数数据等
Model.relax ()	创建一个 MIP 模型的松弛模型

续表

函 数 名	功 能
Model.remove (items)	从模型中移除决策变量、线性约束、二次约束、SOS 约束或者广义约束等
Model.reset (clearall=0)	重置模型至未求解的状态，删除所有之前计算遗留的解的信息
Model.resetParams ()	重置模型的所有参数至默认值
Model.setAttr (attrname, objects, newvalues)	更改一个属性的值
Model.setMObjective (Q, c, constant, xQ_L=None, xQ_R=None, xc=None, sense=None)	用矩阵的形式设置模型的目标函数为二次表达式或者线性表达式
Model.setObjective (expr, sense=None)	设置模型的目标函数为二次表达式或者线性表达式
Model.setObjectiveN (expr, index, priority=0, weight=1, abstol=0, reltol=0, name="")	设置模型的其中一个目标函数为一个线性表达式（适用于多个目标函数的情况）
Model.setPWLObj (var, x, y)	为变量设置分段线性目标函数
Model.setParam (paramname, newvalue)	为参数设置新值
Model.terminate ()	生成一个终止当前优化程序的请求，也就是终止优化程序。当优化程序被终止后，模型的求解状态属性值会相应地变成 GRB_INTERRUPTED
Model.tune ()	自动对模型求解进行调优，寻找能够提升效率的参数设置
Model.update ()	更新模型，执行对模型的修改操作
Model.write (filename)	将模型写成一个文件

6.7.3 Var 类和 MVar 类

Var 类和 MVar 类主要用于添加决策变量，其部分函数如表 6.3 所示。

表 6.3 Var 类和 MVar 类的部分函数

函 数 名	功 能
Var.getAttr (attrname)	查询变量的属性值
Var.sameAs (var2)	检查两个变量对象是不是同一个变量
Var.index (var2)	返回变量在模型约束矩阵中的下标。返回 −2 表示变量被移除了，−1 表示该变量不在模型中，如果是大于或等于 0 的值，就是变量的下标
Var.setAttr (attrname, newvalue)	设置变量的属性值
MVar (vars)	MVar 构建器。该函数创建了一系列的 Var 的对象
MVar.copy ()	复制 MVar 对象

续表

函 数 名	功 能
MVar.getAttr (attrname)	获取一个矩阵变量对象的属性值
MVar.setAttr (attrname, newvalue)	设置矩阵变量的属性值
MVar.sum ()	对 MVar 对象中的变量求和，返回一个 MLinExpr 对象

6.7.4 Column 类

一个 Column 对象（列对象）包含一系列的系数以及相应的约束集合。也就是一个变量参与了哪些约束，以及在约束中的系数是多少和在目标函数中的系数是多少。列对象往往都是临时的，为了构建模型临时创建，配合列生成算法等算法来控制求解过程。Column 类的函数如表 6.4 所示。

表 6.4 Column 类的函数

函 数 名	功 能
Column (coeffs=None, constrs=None)	列构造器
Column.addTerms (coeffs, constrs)	向 Column 对象中添加新项（一项或者多项）
Column.clear ()	移除 Column 对象中的所有项
Column.copy ()	复制 Column 对象
Column.getCoeff (i)	提取 Column 对象中下标为 i 的项的系数
Column.getConstr (i)	根据 Column 中下标为 i 的项，提取其所在的约束。也就是提取第 i 行的约束
Column.remove (item)	从 Column 中移除一项
Column.size ()	提取 Column 对象中的项的数量

6.7.5 目标函数

模型目标函数的相关函数如表 6.5 所示。

表 6.5 模型目标函数的相关函数

函 数 名	功 能
Model.getObjective (index=None)	提取模型的目标函数（多个目标函数返回多个目标）
Model.getPWLObj (var)	提取一个变量 var 的分段线性目标函数
Model.setMObjective (Q, c, constant, xQ_L=None, xQ_R=None, xc=None, sense=None)	用矩阵的形式设置模型的目标函数为二次表达式或者线性表达式
Model.setObjective (expr, sense=None)	设置模型的目标函数为二次表达式或者线性表达式

续表

函 数 名	功 能
Model.setObjectiveN (expr, index, priority=0, weight=1, abstol=0, reltol=0, name="")	设置模型的其中一个目标函数为一个线性表达式（适用于多个目标函数的情况）
Model.setPWLObj (var, x, y)	为变量设置分段线性目标函数

6.7.6 表达式

表达式对象中，重载了运算符 >=、<= 和 ==，在表示 ≥、≤ 和 = 时，直接用上述相应的简单的运算符即可连接对象之间的关系。当然，也可以使用 GRB.LESS_EQUAL、GRB.EQUAL 或 GRB.GREATER_EQUAL 来表示不等关系。两种方法都是可以的。

1. 线性表达式

线性表达式的相关函数主要用于构建目标函数和约束，其相关函数如表 6.6 所示。

表 6.6 线性表达式的相关函数

函 数 名	功 能
LinExpr (arg1=0.0, arg2=None)	线性表达式构建器
LinExpr.add (expr, mult=1.0)	将一个线性表达式对象 expr 添加到另一个线性表达式中
LinExpr.addConstant (c)	在线性表达式中添加一个常数
LinExpr.addTerms (coeffs, vars)	向线性表达式中添加新项。可以添加一个或多个
LinExpr.clear ()	清除线性表达式中的所有元素，也就是设置线性表达式为 0
LinExpr.copy ()	复制线性表达式
LinExpr.getConstant ()	从线性表达式中提取常数项
LinExpr.getCoeff (i)	提取线性表达式中第 i 个下标对应的变量的系数
LinExpr.getValue ()	用当前的解计算线性表达式的值
LinExpr.getVar (i)	提取当前线性表达式中下标 i 对应的决策变量
LinExpr.remove (item)	从线性表达式中移除一项
LinExpr.size ()	提取线性表达式中线性项的数量（不包括常数项）
MLinExpr.copy ()	获得线性矩阵表达式的一个副本
MLinExpr.getValue ()	用当前解计算线性矩阵表达式的值

2. 二次表达式

二次表达式与线性表达式的函数有所不同，其相关函数如表 6.7 所示。

3. 广义表达式

广义表达式属于 GenExpr 类，如取最大值、最小值等广义表达式。

表 6.7 二次表达式的相关函数

函 数 名	功 能
QuadExpr (expr = None)	二次表达式构建器
QuadExpr.add (expr, mult=1.0)	向二次表达式 expr 中添加一个表达式
QuadExpr.addConstant (c)	在二次表达式中添加一个常数
QuadExpr.addTerms (coeffs, vars, vars2=None)	向二次表达式中添加一个线性项或者二次项
QuadExpr.clear ()	清除二次表达式中的所有元素，也就是设置二次表达式为 0
QuadExpr.copy ()	复制二次表达式
QuadExpr.getCoeff (i)	提取二次表达式中第 i 个项对应的系数
QuadExpr.getLinExpr ()	提取二次表达式中的线性表达式的部分。一个二次表达式是由一个线性表达式加上一系列的二次项，因此可以只提取线性的部分
QuadExpr.getValue ()	根据当前解计算二次表达式的值
QuadExpr.getVar1 (i)	提取二次表达式中一项的第 1 个变量。例如一项为 $2xy$, 则提取 x
QuadExpr.getVar2 (i)	提取二次表达式中一项的第 2 个变量。例如一项为 $2xy$, 则提取 y
QuadExpr.remove (item)	从二次表达式中移除一项
MQuadExpr.copy ()	获得二次矩阵表达式的一个副本
MQuadExpr.getValue ()	用当前解计算二次矩阵表达式的值

6.7.7 约束类

1. Model 类中有关约束的函数

约束是运筹优化模型中非常重要的组成部分，Gurobi 中约束相关的函数如表 6.8 所示。

表 6.8 约束相关的函数

函 数 名	功 能
Model.addConstr (lhs, sense=None, rhs=None, name="")	在模型对象中添加一条约束
Model.addConstrs (generator, name="")	在模型对象中添加多条约束
Model.addGenConstrXxx ()	向模型对象中添加广义约束（具体函数名如下）
addGenConstrMax	添加约束 $y = \max\{x_1, x_2, \cdots, c\}$
addGenConstrMin	添加约束 $y = \min\{x_1, x_2, \cdots, c\}$
addGenConstrAbs	添加约束 $y = \lvert x \rvert$
addGenConstrAnd	添加约束 $y = x_1 \wedge x_2 \wedge \cdots$
addGenConstrOr	添加约束 $y = x_1 \vee x_2 \vee \cdots$
addGenConstrIndicator	添加约束 $y = 1 \rightarrow ax \leqslant b$ （an indicator constraint）
addGenConstrPWL	添加约束 $y = \mathrm{PWL}(x)$ （a piecewise-linear function, specified using breakpoints）

续表

函 数 名	功 能
addGenConstrPoly	添加约束 $y=p_0x^d+p_1x^{d-1}+\cdots+p_{d-1}x+p_d$
addGenConstrExp	添加约束 $y=e^x$
addGenConstrExpA	添加约束 $y=a^x$
addGenConstrLog	添加约束 $y=\ln x$
addGenConstrLogA	添加约束 $y=\log_a x$
addGenConstrPow	添加约束 $y=x^a$
addGenConstrSin	添加约束 $y=\sin x$
addGenConstrCos	添加约束 $y=\cos x$
addGenConstrTan	添加约束 $y=\tan x$
Model.addLConstr (lhs, sense=None, rhs=None, name="")	在模型对象中添加一个线性约束（addLConstr() 比 addConstr() 快很多）
Model.addMConstrs (A, x, sense, b, names="")	以矩阵形式向模型中添加多个线性约束
Model.addMQConstr (Q, c, sense, rhs, xQ_L=None, xQ_R=None, xc=None, name="")	以矩阵形式向模型中添加多个二次约束
Model.addQConstr (lhs, sense=None, rhs=None, name="")	在模型中添加一个二次约束
Model.addRange (expr, lower, upper, name="")	在模型中添加一个范围约束 lb ⩽ expr ⩽ ub
Model.cbCut (lhs, sense, rhs)	在 callback 函数中，向 MIP 中添加一个割平面。（注意：该函数只能在 callback 函数中的 where 参数等于 GRB.Callback.MIPNODE 时才会被激发）
Model.cbLazy (lhs, sense, rhs)	在 callback 函数中向 MIP 模型中加入一个 lazy 约束（惰性约束）
Model.chgCoeff (constr, var, newvalue)	改变模型中一个约束中的一个变量的系数
Model.getCoeff (constr, var)	查询一个变量 var 在线性约束 constr 中的系数
Model.getCol (var)	提取决策变量 var 对应的列，包括目标函数系数和对应列的约束系数，返回一个 Column 对象
Model.getConstrByName (name)	根据约束名提取约束。如果多个约束有相同的名字，则该方法将会任意选择一个返回
Model.getConstrs ()	提取多个约束
Model.getGenConstrXxx (genconstr)	提取广义约束的数据（其中 Xxx 是所有广义约束的类型，调用的时候对应到相应的类型即可，可选类型与之前添加约束的部分的类型相同）
Model.getGenConstrs ()	提取模型中的所有广义约束
Model.getPWLObj (var)	提取一个变量 var 的分段线性目标函数值
Model.getQConstrs ()	提取模型中的所有二次约束

续表

函 数 名	功 能
Model.getQCRow (qconstr)	提取二次约束的左端项表达式，返回一个 QuadExpr 对象
Model.getRow (constr)	提取约束 constr 的左端项表达式，返回一个 LinExpr 对象
Model.getSOS (sos)	提取 SOS 约束 sos 的信息，返回结果为一个 tuple，包含 SOS 类型（1 或 2）、参与 SOS 的变量对象以及对应的 SOS 权重
Model.getSOSs ()	提取模型中的所有 SOS 约束

2. Constr，QConstr，SOS 类中的函数

Gurobi 中与约束相关的类主要包括 Constr，QConstr 和 SOS，其相关函数如表 6.9 所示。

表 6.9 Constr, QConstr, SOS 类中的相关函数

函 数 名	功 能
Constr.getAttr (attrname)	查询约束的属性值
Constr.index	获得一个约束的下标
Constr.sameAs (constr2)	检查两个约束对象是不是同一个约束
Constr.setAttr (attrname, newvalue)	设置一个约束的属性值
QConstr.getAttr (attrname)	获得二次约束的属性值
QConstr.setAttr (attrname, newvalue)	设置二次约束的属性值
SOS.getAttr (attrname)	查询 SOS 约束的属性值
GenConstr.getAttr (attrname)	查询广义约束的属性值
GenConstr.setAttr (attrname, newvalue)	设置广义约束的属性值
LinExpr.getVar (i)	提取当前线性表达式中下标 i 对应的决策变量
LinExpr.remove (item)	从线性表达式中移除一项
LinExpr.size ()	提取线性表达式中线性项的数量（不包括常数项）
MLinExpr.copy ()	获得线性矩阵表达式的一个副本
MLinExpr.getValue ()	用当前解计算线性矩阵表达式的值

3. 广义约束帮助函数

Gurobi 中的广义约束包括 And，Abs，Or，Max 和 Min。这些约束可以通过 Model 类中的 addGenConstrXxx() 来构建，也可以用广义约束帮助函数来构建。广义约束帮助函数如表 6.10 所示。

表 6.10 广义约束帮助函数

函 数 名	功 能
abs_ (variable)	用来设置一个变量的取值是另一个变量取值的绝对值
and_ (variables)	用来设置一个 0-1 决策变量等于一系列 0-1 变量的 AND 逻辑约束
or_ (variables)	用来设置一个 0-1 决策变量等于一系列 0-1 变量的 OR 逻辑约束
max_ (variables)	用来设置一个决策变量等于一系列决策变量或者常数项中的最大值
min_ (variables)	用来设置一个决策变量等于一系列决策变量或者常数项中的最小值

6.7.8 求解

在构建完模型以后，我们就可以调用函数 model.optimize(callback = None) 求解模型。如果用户自己设置了 callback 函数，则需要将 callback 函数名传给 model.optimize (callback) 函数。

另外，在求解过程中，可以通过设置求解信息来加速求解。相关函数如表 6.11 所示。

表 6.11 模型求解相关函数

函 数 名	功 能
Model.cbCut (lhs, sense, rhs)	在 callback 函数中向 MIP 中添加一个割平面。（注意：该函数只能在 callback 函数中的 where 参数等于 GRB.Callback.MIPNODE 时才会被调用）
Model.cbGet (what)	在 user callback 函数中查询优化器相关信息
Model.cbGetNodeRel (vars)	在 callback 函数中获取当前节点的线性松弛的解中决策变量的取值
Model.cbGetSolution (vars)	在 callback 函数中获取 MIP 的当前解（整数解）
Model.cbLazy (lhs, sense, rhs)	在 callback 函数中向 MIP 模型中加入一个 lazy 约束（惰性约束）
Model.cbSetSolution (vars, solution)	将一个已知解或者已知的部分解传给当前模型
Model.cbUseSolution ( )	当使用 cbSetSolution 导入一个解之后，可以调用 cbUseSolution 来计算这个解对应的目标函数
Model.chgCoeff (constr, var, newvalue)	改变模型中一个约束中的一个变量的系数
Model.computeIIS (void)	计算一个 IIS
Model.feasRelaxS (relaxobjtype, minrelax, vrelax, crelax)	修改模型对象以获得一个可行的松弛模型
Model.feasRelax (relaxobjtype, minrelax, vars, lbpen, ubpen, constrs, rhspen)	修改模型对象以获得一个可行的松弛模型

续表

函 数 名	功 能
Model.fixed ()	创建一个固定的 MIP 模型。在该固定的模型中，整数变量被固定为其在 MIP 解中的值，并且连续变量也被固定，以满足 SOS 约束和广义约束。被固定的模型中，没有任何整数约束、SOS 约束和广义约束
Model.getTuneResult ()	提取之前调用 tune 之后的调优结果
Model.optimize (callback=None)	求解模型。如果是连续型模型，则调用单纯形法或者内点法进行求解；如果是 MIP 模型，则调用分支切割算法求解
Model.presolve ()	对模型实行预求解
Model.printQuality ()	打印出解的质量统计信息，包括约束违反、整数违反信息等
Model.printStats ()	打印出模型的统计信息（包括约束和变量、非 0 约束系数的个数和最小及最大的约束系数等）
Model.relax ()	创建一个 MIP 模型的松弛模型
Model.reset (clearall=0)	重置模型至未求解的状态，删除所有之前计算遗留的解的信息
Model.resetParams ()	重置模型的所有参数至默认值
Model.setParam (paramname, newvalue)	为参数设置新值
Model.terminate ()	生成一个终止当前优化程序的请求，也就是终止优化程序。当优化程序被终止后，模型的求解状态属性值会相应地变成 GRB_INTERRUPTED
Model.tune ()	自动对模型求解进行调优，寻找能够提升效率的参数设置

6.7.9 解的输出

求解的结果是作为模型中各个对象的属性存储的。模型的目标函数值是 Model 对象的属性，决策变量的取值是 Var 对象的属性，约束的松弛变量和对偶变量是 Constr 对象的属性。模型的求解状态也是 Model 对象的属性。用户只需要清楚自己需要的属性都是哪些对象的属性，就可以用相应的方法获得。

具体的属性列表见本书 6.8.2 节中属性部分的详细介绍。

6.8 Python 接口中的 GRB 类

6.8.1 GRB 类中的常量

下面的常量在引用时只需要加上前缀 GRB. 即可，如 GRB.OPTIMAL，即可获得相应的值。

Gurobi Constants

```
# Status codes
        LOADED = 1
        OPTIMAL = 2
        INFEASIBLE = 3
        INF_OR_UNBD = 4
        UNBOUNDED = 5
        CUTOFF = 6
        ITERATION_LIMIT = 7
        NODE_LIMIT = 8
        TIME_LIMIT = 9
        SOLUTION_LIMIT = 10
        INTERRUPTED = 11
        NUMERIC = 12
        SUBOPTIMAL = 13
        INPROGRESS = 14
        USER_OBJ_LIMIT = 15

# Batch status codes
        BATCH_CREATED = 1
        BATCH_SUBMITTED = 2
        BATCH_ABORTED = 3
        BATCH_FAILED = 4
        BATCH_COMPLETED = 5

# Version number
        VERSION_MAJOR = 9
        VERSION_MINOR = 0
        VERSION_TECHNICAL = 1

# Basis status
        BASIC = 0
        NONBASIC_LOWER = -1
        NONBASIC_UPPER = -2
        SUPERBASIC = -3

# Constraint senses
        LESS_EQUAL = '<'
        GREATER_EQUAL = '>'
        EQUAL = '='

# Variable types
```

```
        CONTINUOUS = 'C'
        BINARY = 'B'
        INTEGER = 'I'
        SEMICONT = 'S'
        SEMIINT = 'N'

# Objective sense
        MINIMIZE = 1
        MAXIMIZE = -1

# SOS types
        SOS_TYPE1 = 1
        SOS_TYPE2 = 2

# General constraint types
        GENCONSTR_MAX = 0
        GENCONSTR_MIN = 1
        GENCONSTR_ABS = 2
        GENCONSTR_AND = 3
        GENCONSTR_OR = 4
        GENCONSTR_INDICATOR = 5
        GENCONSTR_PWL = 6
        GENCONSTR_POLY = 7
        GENCONSTR_EXP = 8
        GENCONSTR_EXPA = 9
        GENCONSTR_LOG = 10
        GENCONSTR_LOGA = 11
        GENCONSTR_POW = 12
        GENCONSTR_SIN = 13
        GENCONSTR_COS = 14
        GENCONSTR_TAN = 15

# Numeric constants
        INFINITY = 1 e100
        UNDEFINED = 1 e101
        MAXINT = 2000000000

# Limits
        MAX_NAMELEN = 255
        MAX_STRLEN = 512
        MAX_TAGLEN = 10240
        MAX_CONCURRENT = 64
```

```
84
85 # Other constants
86         DEFAULT_CS_PORT = 61000
87
88 # Errors
89         ERROR_OUT_OF_MEMORY = 10001
90         ERROR_NULL_ARGUMENT = 10002
91         ERROR_INVALID_ARGUMENT = 10003
92         ERROR_UNKNOWN_ATTRIBUTE = 10004
93         ERROR_DATA_NOT_AVAILABLE = 10005
94         ERROR_INDEX_OUT_OF_RANGE = 10006
95         ERROR_UNKNOWN_PARAMETER = 10007
96         ERROR_VALUE_OUT_OF_RANGE = 10008
97         ERROR_NO_LICENSE = 10009
98         ERROR_SIZE_LIMIT_EXCEEDED = 10010
99         ERROR_CALLBACK = 10011
100        ERROR_FILE_READ = 10012
101        ERROR_FILE_WRITE = 10013
102        ERROR_NUMERIC = 10014
103        ERROR_IIS_NOT_INFEASIBLE = 10015
104        ERROR_NOT_FOR_MIP = 10016
105        ERROR_OPTIMIZATION_IN_PROGRESS = 10017
106        ERROR_DUPLICATES = 10018
107        ERROR_NODEFILE = 10019
108        ERROR_Q_NOT_PSD = 10020
109        ERROR_QCP_EQUALITY_CONSTRAINT = 10021
110        ERROR_NETWORK = 10022
111        ERROR_JOB_REJECTED = 10023
112        ERROR_NOT_SUPPORTED = 10024
113        ERROR_EXCEED_2B_NONZEROS = 10025
114        ERROR_INVALID_PIECEWISE_OBJ = 10026
115        ERROR_UPDATEMODE_CHANGE = 10027
116        ERROR_CLOUD = 10028
117        ERROR_MODEL_MODIFICATION = 10029
118        ERROR_CSWORKER = 10030
119        ERROR_TUNE_MODEL_TYPES = 10031
120        ERROR_SECURITY = 10032
121        ERROR_NOT_IN_MODEL = 20001
122        ERROR_FAILED_TO_CREATE_MODEL = 20002
123        ERROR_INTERNAL = 20003
```

6.8.2 GRB 类中的属性：GRB.Attr

1. Model 对象的属性

Gurobi 中 Model 对象的属性包含模型的所有信息，可以通过属性名获取相应属性的值，从而提取模型的信息。模型对象的属性如表 6.12 所示。

表 6.12 模型对象的属性

属 性 名	属 性 描 述
NumVars	变量的个数
NumConstrs	线性约束的个数
NumSOS	SOS 约束的个数
NumQConstrs	二次约束的个数
NumGenConstrs	广义约束的数量
NumNZs	约束矩阵中非 0 系数的个数
DNumNZs	约束矩阵中非 0 系数的个数（double 类型）
NumQNZs	二次目标函数中，非零项的个数
NumQCNZs	二次约束中，非零项的个数
NumIntVars	整数变量的个数
NumBinVars	0-1 变量的个数
NumPWLObjVars	具有分段线性目标函数的变量的个数
ModelName	模型名称
ModelSense	模型符号（最大化还是最小化）
ObjCon	目标函数中的常数项取值
ObjVal	当前解对应的目标函数值
ObjBound	当前最好的目标函数界限（最小化问题是下界，最大化问题是上界）
ObjBoundC	当前最好的未经圆整的目标函数界限（最小化问题是下界，最大化问题是上界）
PoolObjBound	不在解池中的最好的目标函数界限（最小化问题是下界，最大化问题是上界）
PoolObjVal	在求解过程中存储的所有解中最好的目标函数值
MIPGap	当前的最优 MIP 间隙，也就是当前 gap
Runtime	最近求解的模型的求解时间
Status	当前的模型求解状态
SolCount	存储的解的数量
SolCount	求解过程中存储的解的数量
IterCount	最近求解的模型中，单纯形法迭代的次数
BarIterCount	最近求解的模型中，内点法迭代的次数
NodeCount	最近求解的模型中，branch-and-cut 算法中探索的节点的个数
IsMIP	标识该模型是否是 MIP
IsQP	标识该模型是否是 QP
IsQCP	标识该模型是否是 QCP

续表

属 性 名	属 性 描 述
IsMultiObj	标识该模型是否有多个目标函数
IISMinimal	标识当前的 IIS 是否是最小的
MaxCoeff	约束矩阵中最大的系数（绝对值）
MinCoeff	约束矩阵中最小的非零系数（绝对值）
MaxBound	取值有限的变量中的最大的变量界限值
MinBound	取值有限的变量中的最小的变量界限值
MaxObjCoeff	最大的线性目标函数系数（绝对值）
MinObjCoeff	最小的非零线性目标函数系数（绝对值）
MaxRHS	约束右端常数项的最大值（绝对值）
MinRHS	约束右端常数项的最小非零值（绝对值）
MaxQCCoeff	二次约束的二次部分中，最大的二次约束矩阵系数（绝对值）
MinQCCoeff	二次约束的二次部分中，最小的二次约束矩阵非零系数（绝对值）
MaxQCLCoeff	二次约束的线性部分中，最大的约束矩阵系数（绝对值）
MinQCLCoeff	二次约束的线性部分中，最小的约束矩阵非零系数（绝对值）
MaxQCRHS	最大的二次约束右端项（绝对值）
MinQCRHS	最小的二次约束非零右端项（绝对值）
MaxQObjCoeff	最大的二次目标函数系数（绝对值）
MinQObjCoeff	最小的二次目标函数非零系数（绝对值）
Kappa	近似条件的数量
KappaExact	精确条件的数量
FarkasProof	在 Farkas 不可行证明中的不可行程度
TuneResultCount	用调优工具找到可以提升的参数集
NumStart	MIP 初始解的个数
LicenseExpiration	序列号过期的时间
Server	服务器的名称

2. Var 对象的属性

Var 对象的属性主要包括变量的上界、下界、目标系数、变量类型等。变量对象的属性如表 6.13 所示。

表 6.13 变量对象的属性

属 性 名	属 性 描 述
LB	变量的下界
UB	变量的上界
Obj	变量在线性目标函数中的系数
VType	变量的类型（连续型、0-1 型、整数型等）
VarName	变量名
VTag	变量标签

续表

属 性 名	属 性 描 述
X	变量在当前解中的取值
Xn	变量在次优 MIP 解中的取值
RC	变量的检验数
BarX	变量在最好的内点法迭代中的取值（在 crossover 之前）
Start	MIP 的开始值（为了构建一个初始的 MIP 解）
VarHintVal	MIP 提示值
VarHintPri	MIP 提示优先级
BranchPriority	分支优先级
Partition	可变分区（动态分区）
VBasis	基的状态
PStart	单纯形的开始向量
IISLB	标识在 IIS 中，上界是否有参与
IISUB	标识在 IIS 中，下界是否有参与
PWLObjCvx	标识一个变量是否有凸的分段线性目标函数
SAObjLow	目标函数系数敏感性分析信息（Low）
SAObjUp	目标函数系数敏感性分析信息（Up）
SALBLow	下界敏感性分析信息（Low）
SALBUp	下界敏感性分析信息（Up）
SAUBLow	上界敏感性分析信息（Low）
SAUBUp	上界敏感性分析信息（Up）
UnbdRay	无界射线或极射线

3. LinExpr 对象的属性

LinExpr 对象的线性约束的属性如表 6.14 所示。

表 6.14 线性约束的属性

属 性 名	属 性 描 述
Sense	约束的符号（<,>,=）
RHS	约束的右端项值
ConstrName	约束的名称
CTag	约束的标签
Pi	约束的对偶变量（也就是影子价格）
Slack	针对当前解，该约束对应的松弛变量
CBasis	约束的基状态
DStart	单纯形法开始向量
Lazy	决定一个约束是否要被作为惰性约束
IISConstr	标识在 IIS 中，该约束是否有参与
SARHSLow	右端项的敏感性分析信息（Low）

续表

属 性 名	属 性 描 述
SARHSUp	右端项的敏感性分析信息（Up）
FarkasDual	Farkas 不可行性证明，可作为对偶问题的极射线

4. SOS 对象的属性

SOS 对象的属性如表 6.15 所示。

表 6.15 SOS 对象的属性

属 性 名	属 性 描 述
IISSOS	标识在 IIS 中，该 SOS 约束是否有参与

5. 二次约束对象的属性

二次约束对象的属性如表 6.16 所示。

表 6.16 二次约束对象的属性

属 性 名	属 性 描 述
QCSense	约束的符号（<,>,=）
QCRHS	二次约束的右端项
QCName	二次约束的约束名
QCPi	二次约束的对偶变量值
QCSlack	二次约束针对当前解的松弛变量
QCTag	二次约束的标签
IISQConstr	标识在 IIS 中，该二次约束是否有参与

6. 广义约束对象的属性

广义约束对象的属性如表 6.17 所示。

表 6.17 广义约束对象的属性

属 性 名	属 性 描 述
FuncPieceError	分段线性转化的误差允许值
FuncPieceLength	分段线性转化的分段长度
FuncPieceRatio	控制分段线性转化过程中，采取过低评估策略还是过高评估策略
FuncPieces	分段线性转化的逼近策略
GenConstrType	广义约束的类型
GenConstrName	广义约束的名称
IISGenConstr	标识在 IIS 中，该广义约束是否有参与

7. 解的质量的属性

解的质量的属性如表 6.18 所示。

表 6.18 解的质量的属性

属 性 名	属 性 描 述
BoundVio	最大界限违背（未处理的）
BoundSVio	最大界限违背（处理后的）
BoundVioIndex	违背界限程度最大的变量的索引（未处理的）
BoundSVioIndex	违背界限程度最大的变量的索引（处理后的）
BoundVioSum	界限违背的和（未处理的）
BoundSVioSum	界限违背的和（处理后的）
ConstrVio	最大约束违背（未处理的）
ConstrSVio	最大约束违背（处理后的）
ConstrVioIndex	违背程度最大的约束的索引（未处理的）
ConstrSVioIndex	违背程度最大的约束的索引（处理后的）
ConstrVioSum	约束违背的和（未处理的）
ConstrSVioSum	约束违背的和（处理后的）
ConstrResidual	最大原约束误差（未处理的）
ConstrSResidual	最大原约束误差（处理后的）
ConstrResidualIndex	原约束误差最大的约束的索引（未处理的）
ConstrSResidualIndex	原约束误差最大的约束的索引（处理后的）
ConstrResidualSum	原约束误差的和（未处理的）
ConstrSResidualSum	原约束误差的和（处理后的）
DualVio	最大检验数违背（未处理的）
DualSVio	最大检验数违背（处理后的）
DualVioIndex	检验数违背最大的变量的索引（未处理的）
DualSVioIndex	检验数违背最大的变量的索引（处理后的）
DualVioSum	检验数违背之和（未处理的）
DualSVioSum	检验数违背之和（处理后的）
DualResidual	最大对偶约束误差（未处理的）
DualSResidual	最大对偶约束误差（处理后的）
DualResidualIndex	对偶约束误差最大的变量的索引（未处理的）
DualSResidualIndex	对偶约束误差最大的变量的索引（处理后的）
DualResidualSum	对偶约束误差之和（未处理的）
DualSResidualSum	对偶约束误差之和（处理后的）
ComplVio	最大互补违背
ComplVioIndex	互补违背最大的变量的索引
ComplVioSum	互补违背之和

续表

属 性 名	属性描述
IntVio	最大整数违背
IntVioIndex	最大整数违背的变量的索引
IntVioSum	整数违背之和

8. 多目标函数的属性

多目标函数的属性如表 6.19 所示。

表 6.19 多目标函数的属性

属 性 名	属性描述
ObjN	多目标的目标函数
ObjNCon	多目标的常数项
ObjNPriority	多目标的优先级
ObjNWeight	多目标的权重
ObjNRelTol	多目标的相对容差
ObjNAbsTol	多目标的绝对容差
ObjNVal	多目标目标函数的值
ObjNName	多目标目标函数的名称
NumObj	多目标目标函数的个数

6.8.3 GRB 类中的参数：GRB.Param

1. 模型终止参数

模型终止参数可以控制模型求解的终止。如果在算法的进程中，任意一个参数超过了设定的限制，优化程序将会终止，并且返回一个非最优的求解状态，关于状态的详细解释，见 Gurobi 用户手册的 Status Code 部分。另外，算法终止时，不一定会达到设定的参数限制。模型终止参数如表 6.20 所示。

表 6.20 模型终止参数

参 数 名	功 能
BarIterLimit	内点法迭代次数限制
BestBdStop	最好的停止求解的目标函数界限
BestObjStop	最好的停止求解的目标函数值
Cutoff	目标函数剪枝值，坏于该值的解都将被剪枝
IterationLimit	单纯形法迭代次数限制
NodeLimit	MIP 分支节点数量限制
SolutionLimit	MIP 可行解数量限制
TimeLimit	求解时间限制

2. 容差参数

容差参数是控制当前解与最优解之间的 gap 的边界值，如表 6.21 所示。

表 6.21 容差参数

参 数 名	功 能
BarConvTol	内点法收敛容差
BarQCPConvTol	QCP 内点法收敛容差
FeasibilityTol	原问题可行性容差
IntFeasTol	整数可行性容差
MarkowitzTol	主元阈值容差
MIPGap	MIP 最优性相对间隙
MIPGapAbs	MIP 最优性绝对间隙
OptimalityTol	对偶可行性容差
PSDTol	半正定性容差

3. 单纯形法参数

单纯形法参数用来控制单纯形算法的运算，如表 6.22 所示。

表 6.22 单纯形法参数

参 数 名	功 能
InfUnbdInfo	生成非可行/无界模型的额外信息
NormAdjust	单纯形法 pricing 范数
ObjScale	目标函数处理
PerturbValue	单纯形法扰动程度
Quad	单纯形法中的二次精确度
ScaleFlag	模型处理
Sifting	对偶单纯形法中的筛选
SiftMethod	筛选求解子问题的线性规划方法
SimplexPricing	单纯形法中决策变量的 pricing 策略

4. 内点法参数

内点法参数如表 6.23 所示。

表 6.23 内点法参数

参 数 名	功 能
BarCorrectors	中央校正限制
BarHomogeneous	障碍同质算法
BarOrder	障碍排序算法

续表

参 数 名	功 能
Crossover	障碍交叉策略
CrossoverBasis	交叉初始基构建策略
QCPDual	计算二次约束规划模型的对偶变量

5. MIP 参数

MIP 参数如表 6.24 所示。

表 6.24 MIP 参数

参 数 名	功 能
BranchDir	分支方向预设值
ConcurrentJobs	启用分布式并发求解器
ConcurrentMIP	启用并发 MIP 求解器
ConcurrentSettings	逗号分隔.prm 文件列表，用于创建并发
ConcurrentSettings	逗号分隔.prm 文件列表，用于创建并发环境
DegenMoves	单纯形法退化移动
Disconnected	断开组件连接策略
DistributedMIPJobs	启用分布式 MIP 求解器
Heuristics	打开或者关闭 MIP 启发式
ImproveStartGap	解改进开关，开始间隙
ImproveStartNodes	解改进开关，开始节点
ImproveStartTime	解改进开关，开始时间
LazyConstraints	添加惰性约束必须要设置的参数
MinRelNodes	最小松弛启发式控制
MIPFocus	设置 MIP 求解器的侧重点
MIQCPMethod	求解 MIQCP 的算法
NodefileDir	MIP 节点文件的文件目录
NodefileStart	导出 MIP 分支树信息到硬盘的内存阈值
NodeMethod	求解 MIP 节点线性松弛问题的算法
NonConvex	控制如何处理非凸二次规划问题
PartitionPlace	控制分割启发式运行的时机
PumpPasses	可行性泵启发式控制
RINS	松弛诱导的邻域搜索
SolFiles	存储中间解的文件位置
SolutionNumber	次优 MIP 解的提取
StartNodeLimit	MIP 开始解的子 MIP 的节点数量
StartNumber	设置 MIP 开始解的索引
SubMIPNodes	子 MIP 启发式探索的节点

续表

参 数 名	功 能
Symmetry	MIP 对称检测
VarBranch	分支变量选择策略
ZeroObjNodes	零目标函数启发式控制

6. 预求解参数

预求解参数控制的是预求解算法的运算，如表 6.25 所示。

表 6.25 预求解参数

参 数 名	功 能
AggFill	预处理聚合过程中允许填充
Aggregate	预处理聚合控制
DualReductions	在预处理过程中禁用对偶缩减
PreCrush	允许预处理将原始模型上的约束转换为对预处理模型上的等效约束
PreDepRow	预处理依赖行减少
PreDual	预处理对偶
PreMIQCPForm	预处理的 MIQCP 模型格式
PrePasses	预处理传参限制
PreQLinearize	预处理 Q 矩阵线性化
Presolve	预处理的等级
PreSOS1BigM	控制 SOS1 转换为二进制形式
PreSOS2BigM	控制 SOS2 转换为二进制形式
PreSparsify	控制预处理稀疏性的缩减

7. 调优参数

Gurobi 的调优参数如表 6.26 所示。

表 6.26 调优参数

参 数 名	功 能
TuneCriterion	指定调参准则
TuneJobs	指定分布式调优的工作的数量
TuneOutput	调整输出水平
TuneResults	返回改进参数集的数量
TuneTimeLimit	调参时间限制
TuneTrials	对每个参数集运行多次以限制随机噪声的影响

8. 多个解的参数

多个解的参数如表 6.27 所示。

表 6.27 多个解的参数

参 数 名	功 能
PoolGap	在指定调整标准下池中解的差距
PoolSearchMode	选择用于查找其他解的方法
PoolSolutions	保留在池中的解的数量

9. MIP 割参数

Gurobi 的割平面的相关参数如表 6.28 所示，调整这些参数可以控制 Gurobi 在求解 MIP 过程中是否使用相应的割平面。

表 6.28 MIP 割参数

参 数 名	功 能
BQPCuts	布尔二次曲面割生成
Cuts	全局割生成控制
CliqueCuts	团割生成
CoverCuts	覆盖割生成
CutAggPasses	割生成期间执行的约束聚合传参
CutPasses	根割平面传参次数限制
FlowCoverCuts	流覆盖割生成
FlowPathCuts	流路径割生成
GomoryPasses	根 Gomory 割传参次数限制
GUBCoverCuts	广义上界覆盖割生成
ImpliedCuts	隐边界切割生成
InfProofCuts	不可行证明割生成
MIPSepCuts	MIP 分离割生成
MIRCuts	混合整数圆整割生成
ModKCuts	Mod-k 割生成
NetworkCuts	网络割生成
ProjImpliedCuts	投影边界割生成
RelaxLiftCuts	松弛-提升割生成
RLTCuts	重构线性化技巧割生成
StrongCGCuts	强 Chvátal-Gomory 割生成
SubMIPCuts	子 MIP 割生成
ZeroHalfCuts	零-半割生成

10. 其他参数

Gurobi 的一些其他参数，包括求解信息展示参数、随机数种子参数等，具体如表 6.29 所示。

表 6.29 其他参数

参 数 名	功 能
DisplayInterval	打印日志行的频率
FeasRelaxBigM	保证可行松弛的大 M 数值
FuncPieceError	函数约束的分段线性函数转换所允许的误差
FuncPieceLength	函数约束的分段线性函数转换的片段长度
FuncPieceRatio	控制在分段线性函数近似中是低估还是高估函数值
FuncPieces	设置分段线性近似函数逼近的策略
FuncMaxVal	函数约束中 x 和 y 变量的最大值
IgnoreNames	指示是否忽略用户提供的名称
IISMethod	IIS 方法
InputFile	优化开始之前要读取的文件
JSONSolDetail	控制存储在生成的 JSON 解中的详细程度
LogFile	日志文件名
LogToConsole	控制台记录
Method	用于求解连续模型的算法
MultiObjMethod	热启动方法以解决后续目标函数
MultiObjPre	初步预处理多目标模型
NumericFocus	设定数值侧重点
ObjNumber	设置多目标索引
OutputFlag	求解器输出控制
Record	启用 API 调用记录
ResultFile	完成优化后写入结果文件
ScenarioNumber	在多场景模型中设置场景索引
Seed	修改随机数种子
Threads	要启用的并行线程数
UpdateMode	更改惰性更新

6.9 Gurobi 的回调函数

6.9.1 什么是 Gurobi 的回调函数

callback 函数是 Gurobi 为用户提供的高级控制功能，它使得用户在 Gurobi 对问题求解过程中可以进行信息获取、加入新约束或自己开发的算法、终止求解过程等操作，也

就是说 Gurobi Callback 可以用来对问题的优化过程进行查询或改变优化过程。简言之，callback 是 Gurobi 为用户提供的用来监控、修改优化状态的函数。

Gurobi Callback 提供了两个重要的参数：where 和 what。where 是回调函数触发点，表明了用户在哪里进行 callback 的调用。what 是要进行的操作，即能够获取什么信息，这里注意获取的信息类型取决于 where。参数 where 的取值如表 6.30所示。

表 6.30 参数 where 的取值

where	数 值	优化器状态
POLLING	0	周期性轮询回调
PRESOLVE	1	预处理
SIMPLEX	2	单纯形
MIP	3	当前 MIP
MIPSOL	4	发现新的 MIP 解
MIPNODE	5	当前探索节点
MESSAGE	6	打印出日志信息
BARRIER	7	当前内点法
MULTIOBJ	8	当前多目标

由于 what 取值取决于 where 取值，因此使用时要注意正确对应两者的关系，否则将会出错。比如当 where=MIP 时，what 取值如表 6.31所示。

表 6.31 当参数 where=MIP 时，参数 what 的可选取值

what	类 型	描 述
MIP_OBJBST	double	当前最优目标值
MIP_OBJBND	double	当前最优界
MIP_NODCNT	double	当前已搜索的节点数
MIP_SOLCNT	int	当前发现可行解的数量
MIP_CUTCNT	int	当前割平面使用次数
MIP_NODLFT	double	当前未搜索的节点数
MIP_ITRCNT	double	当前单纯形迭代步数

再如当 where=MIPSOL 时，what 的取值如表 6.32所示。

表 6.32 当参数 where=MIPSOL 时，参数 what 的取值

what	类 型	描 述
MIPSOL_SOL	double	当前解的具体取值
MIPSOL_OBJ	double	新解的目标值

续表

what	类　型	描　述
MIPSOL_OBJBST	double	当前最优目标值
MIPSOL_OBJBND	double	当前最优界
MIPSOL_NODCNT	double	当前已搜索的节点数
MIPSOL_SOLCNT	int	当前发现可行解的数量

6.9.2 Gurobi 回调函数的用法

一般通过 model 对象给 callback 函数传递数据，当使用多个线程求解一个模型时，callback 只会从一个线程中调用，所以不需要担心回调的线程安全问题。此外，Gurobi 不支持在 callback 内更改参数，这样做可能会导致未定义的行为。Python 调用 Gurobi Callback 的例子在 Gurobi 文件的 callback.py 文件中，详细例子请读者自行查阅，本节只介绍几种常用的用法，主要参考文献为 Gurobi 官方文档。

callback 函数查询信息：可以查询目标值、节点数等。比如查询当前单纯形的目标函数值（where==SIMPLEX），代码如下。

CallbackQueryObj.py

```
/**
* @author: Yongsen Zang
* @School: Tsinghua University
* @操作说明：代码参考文献为Gurobi官方文档
*
*/
def mycallback(model, where):
 if where == GRB.Callback.SIMPLEX:
    print(model.cbGet(GRB.Callback.SPX_OBJVAL))
model.optimize(mycallback)
```

再比如查询当前节点的松弛解（where=MIPNODE），代码如下。

CallbackQueryNodeStatus.py

```
def mycallback(model, where):
    if where == GRB.Callback.MIPNODE and model.cbGet(GRB.Callback.MIPNODE_
      STATUS)== GRB.OPTIMAL :
       print model.cbGetNodeRel(model._vars)
model._vars = model.getVars()
model.optimize(mycallback)
```

再比如查询可行解的取值（where=MIPSOL），代码如下。注意，这里一定要在调用 callback 函数之前为模型定义外部变量，否则在 callback 函数中可能会出现不能识别变量的情况。

CallbackQuerySOL.py

```
def mycallback(model, where):
 if where == GRB.Callback.MIPSOL:
    print (model.cbGetSolution(model._vars))
model._vars = model.getVars()
model.optimize(mycallback)
```

再诸如通过节点的松弛解信息构造割平面（where=MIPNODE），代码如下。这里一定要注意设置模型参数 PreCrush 的值为 1，否则无法成功添加用户自定义割平面。

CallbackQueryNodeStatus.py

```
def mycallback(model, where):
 if where == GRB.Callback.MIPNODE:
    status = model.cbGet(GRB.Callback.MIPNODE_STATUS)
    if status == GRB.OPTIMAL:
       rel = model.cbGetNodeRel([model._vars[0], model._vars[1]])
          if rel[0] + rel[1] > 1.1:
             model.cbCut(model._vars[0] + model._vars[1] <= 1)
model._vars = model.getVars()
model.Params.PreCrush = 1
model.optimize(mycallback)
```

通过可行解的信息构造 Lazy Cut（where=MIPSOL），代码如下。注意这里一定要先打开惰性约束，然后再求解模型，否则会报错。

CallbackQueryNodeStatus.py

```
def mycallback(model, where):
   if where == GRB.Callback.MIPSOL:
      sol = model.cbGetSolution([model._vars[0], model._vars[1]])
          if sol[0] + sol[1] > 1.1:
             model.cbLazy(model._vars[0] + model._vars[1] <= 1)
model._vars = model.getVars()
model.Params.lazyConstraints = 1
model.optimize(mycallback)
```

6.10 Python 调用 Gurobi 的参数调优

在 Gurobi 中可以通过参数调优的工具来探索不同的参数组合，以提升求解效率。详细操作可参考http://www.gurobi.cn/。调用自动参数调优的方法有以下两种。

（1）通过命令行 grbtune TuneTimeLimit=100 C:\gurobi801\win64\examples\data\misc07.mps 进行。

(2) 通过 API 进行：首先使用 `Model.resetParams()` 清除已设参数；然后设置自动调优时间 `Model.Params.TuneTimeLimit` 等调优的参数；最后运行调优工具 `Model.tune()`，Gurobi 就会在调优时间内搜索并显示较好的参数组合。

自动调优参数如表 6.33 所示。

表 6.33 自动调优参数

参 数 名	作 用	取 值
TuneCriterion	调整参数的准则	−1：默认，缩短发现最优解所需的时间；1：最优的 Gap；2：最好的可行解；3：最好的 bound
TuneJobs	分布式并行调参	0：默认
TuneOutput	控制输出结果的量	0：没有输出；1：发现最好的参数组合时输出；2：默认，输出试过的参数组合；3：试过的参数组合和详细的求解器输出
TuneResults	返回最优参数组合的数量	−1：默认，按照调整参数的个数返回调参结果
TuneTimeLimit	调参时间	−1：默认，自动选择时间，以秒为单位
TuneTrials	每组参数组合的运行次数	主要目的是减小随机因素的影响

6.11 Python 调用 Gurobi 求解整数规划的简单例子

假设有下面的整数规划问题：

$$\min \sum_{i=1}^{10} \left(5x_i^1 + 4x_i^2 + 3x_i^3 + 2x_i^4 + x_i^5\right) \tag{6.10}$$

$$10x_i^1 + 7x_i^2 + 5x_i^3 + 4x_i^4 + x_i^5 = D_i \tag{6.11}$$

$$x_i^1 + x_i^2 + x_i^3 + x_i^4 + x_i^5 \leqslant D_i \tag{6.12}$$

$$x_i^1 + 2x_i^2 + 3x_i^3 + 6x_i^4 + 10x_i^5 \leqslant \mathrm{INC}_i \tag{6.13}$$

$$x_i^1, x_i^2, x_i^3, x_i^4, x_i^5 \geqslant 0 \text{ and integer}, \forall i \in \{1, 2, \cdots, 10\} \tag{6.14}$$

下面我们用 Python 调用 Gurobi 求解该问题。具体参数 D_i, INC_i 见如下代码：

Gurobi Python Interface

```
# coding: utf-8

# import these packages before coding
from gurobipy import *
import pandas as pd
import numpy as np

```

```
# These input parameters can be obtained by reading. txt file etc.
D = [60, 60, 60, 72, 72, 82, 60, 80, 80, 90]
INC = [10, 20, 30, 60, 80, 50, 60, 70, 90, 80]
obj_coef = [5, 4, 3, 2, 1]

coef = [[10, 7, 5, 4, 1],
        [1, 1, 1, 1, 1],
        [1, 2, 3, 6, 10]]


# construct model object
model = Model('IP_example')

# introduce decision variable by cycle
x = [[[] for i in range(5)] for j in range(len(D))]
for i in range(len(D)):
    for j in range(5):
        x[i][j] = model.addVar(lb = 0.0     # lower bound
                               ,ub = 100000          #upper bound
                               ,vtype = GRB.INTEGER #decision variable type
                               ,name = "x_" + str(i) + "_" + str(j)
                               )

# objective function
obj = LinExpr(0)

for i in range(len(D)):
    for j in range(5):
        obj.addTerms(obj_coef[j], x[i][j])

model.setObjective(obj, GRB.MINIMIZE)

# Constraint 1
for i in range(len(D)):
    expr = LinExpr(0)
    for j in range(5):
        expr.addTerms(coef[0][j], x[i][j])
    model.addConstr(expr == D[i], name="D_" + str(i))

# Constraint 2
for i in range(len(D)):
    expr = LinExpr(0)
```

```
    for j in range(5):
        expr.addTerms(coef[1][j], x[i][j])
    model.addConstr(expr <= D[i], name="D_2" + str(i))

# Constraint 3
for i in range(len(D)):
    expr = LinExpr(0)
    for j in range(5):
        expr.addTerms(coef[2][j], x[i][j])
    model.addConstr(expr <= INC[i], name="INC_" + str(i))



# solve the constructed model
model.write('model.lp')
model.optimize()

for var in model.getVars():
    if(var.x > 0):
        print(var.varName, '\t', var.x)
```

求解结果如下：

Gurobi Python Interface

```
Optimal solution found (tolerance 1.00e-04)
Best objective 3.580000000000e+02, best bound 3.580000000000e+02, gap
      0.0000%
x_0_0    6.0
x_1_0    6.0
x_2_0    6.0
x_3_0    4.0
x_3_3    8.0
x_4_0    2.0
x_4_3    13.0
x_5_0    7.0
x_5_3    3.0
x_6_0    6.0
x_7_0    8.0
x_8_0    8.0
x_9_0    9.0
```

第7章　调用CPLEX和Gurobi求解MIP的复杂案例：VRPTW和TSP

前面两章我们介绍了优化求解器的 API 以及一些简单的案例。但是在实际的生产和科研中，问题都比较复杂，决策变量的个数和约束的个数都成千上万，对于这种大规模的复杂模型，如何用求解器实现建模及求解？本章就以 VRPTW 为例，讲解一些比较复杂的模型的建模和求解。

TSP 的部分主要展现求解器的高级用法：callback 的使用。特别地，后续章节要介绍的 Branch-and-Cut 等算法的代码实现中也用到了 callback，因此熟练掌握 callback 的使用方法对之后的学习非常有帮助。

7.1　调用 CPLEX 和 Gurobi 求解 VRPTW

7.1.1　VRPTW 的一般模型

前面的章节我们已经给出了 VRPTW 的标准模型，具体如下：

$$\min \quad \sum_{k\in K}\sum_{i\in V}\sum_{j\in V} c_{ij}x_{ijk} \tag{7.1}$$

$$\sum_{k\in K}\sum_{j\in V} x_{ijk} = 1, \qquad \forall i \in C \tag{7.2}$$

$$\sum_{j\in V} x_{0jk} = 1, \qquad \forall k \in K \tag{7.3}$$

$$\sum_{i\in V} x_{ihk} - \sum_{j\in V} x_{hjk} = 0, \qquad \forall h \in C, \forall k \in K \tag{7.4}$$

$$\sum_{i\in V} x_{i,n+1,k} = 1, \qquad \forall k \in K \tag{7.5}$$

$$\sum_{i\in C} q_i \sum_{j\in V} x_{ijk} \leqslant Q, \qquad \forall k \in K \tag{7.6}$$

$$s_{ik} + t_{ij} - M\left(1 - x_{ijk}\right) \leqslant s_{jk}, \qquad \forall (i,j) \in A, \forall k \in K \tag{7.7}$$

$$e_i \leqslant s_{ik} \leqslant l_i, \qquad \forall i \in V, \forall k \in K \tag{7.8}$$

$$x_{ijk} \in \{0,1\}, \qquad \forall (i,j) \in A, \forall k \in K \tag{7.9}$$

其中每个约束的含义见本书第 2 章。

7.1.2 Java 调用 CPLEX 求解 VRPTW

本部分代码主要包含 4 个类：Main、Instance、Node 和 Result。

- Main 类：主函数所在的类。包含读取数据、模型建立及模型求解。Main 类中的函数 double_truncate(double v) 用于截断小数，例如将四位小数 3.6546 截断为 2 位小数 3.65。该函数参考自微信公众号“数据魔术师”共享的代码；
- Instance 类：存储算例数据；
- Node 类：存储每个客户点的各项数据；
- Result 类：存储最优解。

1. Main 类

Main 类代码如下：

Main.java

```
package VRPTW_Hsinglu;

import ilog.concert.IloColumn;

/**
 * @author: Hsinglu Liu
 * @School: Tsinghua University
 * @操作说明：修改读取文件的文件名，求解C101和R101两个算例，
 *          修改customerNum来截取100个客户点中前customerNum个客户点，求解不同规模的VRPTW
 * 算例来源：Solomon于1987年发表在OR上的论文
 *
 */

import ilog.concert.IloException;
import ilog.concert.IloLinearNumExpr;
import ilog.concert.IloNumExpr;
import ilog.concert.IloNumVar;
import ilog.concert.IloNumVarType;
import ilog.concert.IloObjective;
import ilog.concert.IloRange;
import ilog.cplex.IloCplex;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
```

```
import java.util.ArrayList;
import java.util.List;

public class Main {

    //算例截取参数
    //customerNum = 30, 表示截取c101中100个顾客点的前30个顾客点作为测试数据
    static final int customerNum = 25;
    static final int nodeNum = customerNum + 2;  //节点的数量=客户点的数量
          //+2
    static final int vehicleNum = 25;
    static double gap = 1e-8;
    public static void main(String[] args) throws IOException, IloException
       {
        //首先写入算例的路径
        String path = "E:\\MyCode\\JavaCode\\JavaCallCplex\\src\\VRPTW_
       Hsinglu\\c101.txt";

        //根据算例路径，读取算例数据
        Instance instance = readData(path);

        //打印出算例数据信息
        printInstance(instance);

        //调用CPLEX构建VRPTW模型
        //          IloCplex Cplex = CPLEX.buildModel(instance);
        //          Cplex.setParam(IloCplex.DoubleParam.TimeLimit,20);

        //================== 1. 定义CPLEX对象 ========================
        //只有1步：创建CPLEX模型对象
        /*
         * CPLEX对象包含了模型的所有信息，包括：
         *       决策变量（类型、上界、下界）
         *       约束（类型、系数、左端项、>=/<=/=，右端项）
         *
         * 只需要一步一步循环添加到CPLEX类之中就可以了，最后调用
       cplex.solve()方法进行求解即可
         */
        IloCplex Cplex = new IloCplex();  //这里需要抛异常

        //================== 2. 定义决策变量 ========================
        //包含2步：（1）创建决策变量对象；（2）设置决策变量的类型和上下界
```

```
        /*
         * ilog.concert包中包含了一些跟表达式和数值有关的类
         * 跟决策变量有关的class主要有以下几个
         *              IloIntVar: getMax(), getMin(), setMax(int), setMin(int)
         *              IloNumVar: getLB(), getName(), gettype(), getUB(),
    setLB(), setName(), setUB()
         *
         */

        IloNumVar[][][]X = new IloNumVar[instance.nodeNum][instance.nodeNum]
        [instance.vehicleNum];
        IloNumVar[][] S = new IloNumVar[instance.nodeNum][instance.
    vehicleNum];
        for(int i = 0; i < instance.nodeNum; i++){
            for(int j = 0; j < instance.nodeNum; j++){
                for(int k = 0; k < instance.vehicleNum; k++){
                    if(i == j){
                        //*****************************
                        /*
                         * 注意：这一步很重要，i和j相等时可以置null，当然也
                         * 可以不置空
                         *       或者是X[i][j][k] = null;
                         *       或者是X[i][j] = null;
                         */
                        X[i][j][k] = null;     //可有可无
                        //X[i][j] = null;
                    }else{
                        X[i][j][k] = Cplex.boolVar("X" + i + j + k);
                        //S[i][k] = Cplex.numVar(0, Integer.MAX_VALUE);
                        S[i][k] = Cplex.intVar(0, Integer.MAX_VALUE, "S" + i
    + k);
                    }
                }
            }
        }

        // ================= 3. 定义目标函数 ========================
        //本函数的目的是：建立目标函数表达式，并且将其添加到CPLEX模型中
        /*
         * 首先需要新建一个IloLinearNumExpr类的对象，然后对其赋值，最后添加到
         * CPLEX模型中
         */
```

```
100
101         //IloLinearNumExpr obj = new IloLinearNumExpr();    注意：这种方法是错误的
102         /*
103          * 创建表达式的函数有以下几个
104          * IloLinearNumExpr: add(IloLinearNumExpr)
105          *                   addTerm(double, IloNumVar), addTerm(IloNumVar, double)
106          *                   addTerms(double[], IloNumVar[]), addTerms(IloNumVar[],
                               *double[])
107          *
108          */
109         IloNumExpr obj = Cplex.numExpr();
110         for (int i = 0; i < instance.nodeNum; i++) {
111             for (int j = 0; j < instance.nodeNum; j++) {
112                 if (i != j) {
113                     System.out.println("i = " + i + ", \t j = " + j);
114                     int x1 = instance.Nodes.get(i).Xcoor;
115                     int y1 = instance.Nodes.get(i).Ycoor;
116                     int x2 = instance.Nodes.get(j).Xcoor;
117                     int y2 = instance.Nodes.get(j).Ycoor;
118                     double arcDistance = Math.sqrt((x2 - x1) *
119                     (x2 - x1) + (y2 - y1) * (y2 - y1));
120                     arcDistance = double_truncate(arcDistance);
121                     System.out.println(i + "\t" + j + "\t" + arcDistance);
122
123                     for (int k = 0; k < instance.vehicleNum; k++) {
124                         //构建目标函数
125                         obj = Cplex.sum(obj,
126                                 Cplex.prod(arcDistance, X[i][j][k]));
127                         //obj.addTerm(arcDistance, X[i][j][k]); //即Cij * Xijk
128                     }
129                 }
130             }
131         }
132         Cplex.addMinimize(obj);
133
134         // ================= 4. 定义约束条件 ========================
135         //包括2步：（1）构建线性数值表达式IloLinearNumExpr; (2)加入右端常数和
            //大/小于号addEq/addGe/addLe
136         /*
137         * 添加约束的3个函数：
138         *     addEq():添加等式约束
139         *  addEq(double arg0, IloNumExpr arg1): 例如：  1 = y1
```

```
140         *  addEq(IloNumExpr arg0, double arg1): 例如:  y1 = 1
141         *  addEq(IloNumExpr arg0, IloNumExpr arg1): 例如:  y1 = x1
142         *  addEq(double arg0, IloNumExpr arg1, String arg2): 例如: 1 = y1," 约束 1"
143         *  addEq(IloNumExpr arg0, double arg1, String arg2): 例如: y1 = 1," 约束 1"
144         *  addEq(IloNumExpr arg0,IloNumExpr arg1, String arg2): 例如: y1=x1, " 约束 1"
145         *
146         *      addGe():添加>=约束
147         *
148         *      addLe():添加<=约束
149         */
150
151         //首先引入两个线性表达式
152         IloNumExpr expr = Cplex.numExpr();
153         IloNumExpr expr1 = Cplex.numExpr();
154         IloNumExpr expr2 = Cplex.numExpr();
155         //约束1
156         for(int i = 1; i < instance.nodeNum - 1; i++){
157             expr = Cplex.numExpr();
158             for(int j = 0; j < instance.nodeNum; j++){
159                 for(int k = 0; k < instance.vehicleNum; k++){
160                     if(i != j){  //注意，这里一定要判断i!=j这一项，否则求解
        //不出来
161                         expr = Cplex.sum(expr, X[i][j][k]);
162                     }
163                 }
164             }
165             Cplex.addEq(expr, 1.0);
166         }
167
168         //约束2
169         for (int k = 0; k < instance.vehicleNum; k++) {
170             expr = Cplex.numExpr();
171             for (int i = 1; i <= instance.customerNum; i++) {
172                 for (int j = 0; j < instance.nodeNum; j++) {
173                     if (i != j) { //注意，这里一定要判断i!=j这一项，否则求解
        //不出来
174                         expr = Cplex.sum(expr, Cplex.prod(
175                                 instance.Nodes.get(i).demand, X[i][j][k]));
176                     }
177                 }
178             }
179             Cplex.addLe(expr, instance.vehicleCapacity);
```

```
180         }
181
182         //约束3
183         for(int k = 0; k < instance.vehicleNum; k++){
184             expr = Cplex.numExpr();
185             for(int j = 1; j < instance.nodeNum; j++){
186                 expr = Cplex.sum(expr, X[0][j][k]);
187             }
188             Cplex.addEq(expr, 1);
189         }
190
191         //约束4
192         for(int k = 0; k < instance.vehicleNum; k++){
193             for(int h = 1; h < instance.nodeNum - 1; h++){
194                 expr1 = Cplex.numExpr();
195                 expr2 = Cplex.numExpr();
196                 for(int i = 0; i < instance.nodeNum; i++){
197                     if(i != h){ //注意，这里一定要判断 i!=h 这一项，否则求解
    //不出来
198                         expr1 = Cplex.sum(expr1, X[i][h][k]);
199                     }
200                 }
201                 for(int j = 0; j < instance.nodeNum; j++){
202                     if(h != j){ //注意，这里一定要判断 h!=j 这一项，否则求解
    //不出来
203                         expr2 = Cplex.sum(expr2, X[h][j][k]);
204                     }
205                 }
206                 Cplex.addEq(expr1, expr2);
207             }
208         }
209
210         //约束5
211         for(int k = 0; k < instance.vehicleNum; k++){
212             expr = Cplex.numExpr();
213             for(int i = 0; i < instance.nodeNum - 1; i++){  //注意这里的序号
    //问题
214                 expr = Cplex.sum(expr, X[i][instance.nodeNum - 1][k]);
215             }
216             Cplex.addEq(expr, 1.0);
217         }
218
```

```
219         //约束6
220         int M = Integer.MAX_VALUE;
221         for(int i = 0; i < instance.nodeNum; i++){
222             for(int j = 0; j < instance.nodeNum; j++){
223                 for(int k = 0; k < instance.vehicleNum; k++){
224                     if(i != j){
225                         int x1 = instance.Nodes.get(i).Xcoor;
226                         int y1 = instance.Nodes.get(i).Ycoor;
227                         int x2 = instance.Nodes.get(j).Xcoor;
228                         int y2 = instance.Nodes.get(j).Ycoor;
229                         double arcDistance = Math.sqrt((x2 - x1) * (x2 - x1)
    + (y2 - y1) * (y2 - y1));
230                         arcDistance = double_truncate(arcDistance);
231
232                         expr = Cplex.numExpr();
233                         expr = Cplex.sum(expr, S[i][k]);
234                         expr = Cplex.sum(expr, arcDistance);
235                         expr = Cplex.diff(expr, S[j][k]);
236                         expr = Cplex.diff(expr, M);   //diff就是减法的意思
237                         expr = Cplex.sum(expr, Cplex.prod(M, X[i][j][k]));
238                         Cplex.addLe(expr, 0);
239                     }
240                 }
241             }
242
243         }
244
245         //约束7
246         for(int i = 0; i < instance.nodeNum; i++){
247             for(int k = 0; k < instance.vehicleNum; k++){
248                 expr = Cplex.numExpr();
249                 expr = Cplex.sum(expr1, S[i][k]);
250                 Cplex.addGe(expr, instance.Nodes.get(i).readyTime);
251                 Cplex.addLe(expr, instance.Nodes.get(i).dueTime);
252             }
253         }
254
255         //设置求解时间
256         //Cplex.setParam(IloCplex.DoubleParam.TimeLimit, 50);
            //设置求解的时间限制
257         //Cplex.setParam(IloCplex.DoubleParam.TiLim, 20); //设置求解的时间限制
258         //Cplex.setParam(IloCplex.DoubleParam.EpAGap, 0.88); //设置求解的绝对Gap
```

```
259
260         //求解模型
261         Cplex.solve();
262
263
264         //--------------输出结果-----------------------------
265         System.out.println("\n\n----------以下是求解结果---------\n");
266         System.out.println("Objective\t\t:" + Cplex.getObjValue());
267         int vehicleNum = 0;
268         for (int k = 0; k < instance.vehicleNum; k++) {
269             //System.out.printf("第%5d  辆车: ", k + 1);
270             wc:for (int i = 0; i < instance.nodeNum; i++) {
271                 nc:for (int j = 0; j < instance.nodeNum; j++) {
272                     if (i != j) {
273                         if (Cplex.getValue(X[i][j][k]) != 0 && i != 0 && j !
    = instance.nodeNum - 1) {
274                             vehicleNum += 1;
275                             break wc;
276                         }
277                     }
278                 }
279             }
280
281         }
282         System.out.println("vehicle number\t\t:" + vehicleNum);
283
284         int count = 1;
285         int nextNode = 0;
286         for (int k = 0; k < instance.vehicleNum; k++) {
287             //System.out.print(" 第" + count +
288             //" 辆车");
289             double load = 0;
290             double distance = 0;
291             w:while (nextNode != instance.nodeNum - 1) {
292                 int i = nextNode;
293                 wc: for (; i < instance.nodeNum; i++) {
294                     nc: for (int j = 0; j < instance.nodeNum; j++) {
295                         if (i != j && Cplex.getValue(X[i][j][k]) != 0) {
296                             if (i == 0 && j == instance.nodeNum - 1) {
297                                 break w;
298                             } else if (i != 0 || j != instance.nodeNum - 1)
    {
```

```
                                System.out.print(nextNode + "-");
                                load += instance.Nodes.get(j).demand;

                                int x1 = instance.Nodes.get(i).Xcoor;
                                int y1 = instance.Nodes.get(i).Ycoor;
                                int x2 = instance.Nodes.get(j).Xcoor;
                                int y2 = instance.Nodes.get(j).Ycoor;
                                double arcDistance = Math.sqrt((x2 - x1) *
        (x2 - x1) + (y2 - y1) * (y2 - y1));
                                arcDistance = double_truncate(arcDistance);

                                distance += arcDistance; //计算该条路径的
        //距离
                                nextNode = j;
                                i = nextNode - 1;
                                if(nextNode == instance.nodeNum - 1){
                                    System.out.print(0 + "\t\tCap: " + load
      + "\t" + "distance: "  + distance + "\n");
                                    nextNode = 0;
                                    break w;
                                }
                                break nc;
                            }
                        }
                    }
                }
            }
            // System.out.println();
        }

        //粗略输出的效果
        //      for(int k = 0; k < instance.vehicleNum; k++){
        //          System.out.println("------第" + k + " 车-------");
        //          for(int i = 0; i < instance.nodeNum; i++){
        //              for(int j = 0; j < instance.nodeNum; j++){
        //                  if(i != j && Cplex.getValue(X[i][j][k]) != 0){
        //                  System.out.println("X["+i+","+j+","+k+"]="+1);
        //                  }
        //              }
        //          }
        //      }
    }
```

```
338
339     //读取算例数据的方法
340     public static Instance readData(String path) throws IOException {
341         //首先创建存放所有节点的数据的列表
342
343         Instance instance = new Instance();
344         List<Node> Nodes = new ArrayList<Node>();
345
346         //创建文件读入对象
347         BufferedReader br = new BufferedReader(new FileReader(path));
            //这里需要抛异常
348
349         //进行读取操作
350         String line = null;
351         int count = 0;
352         while((line = br.readLine()) != null){
353             count += 1;
354             System.out.println(line + count);
355             if(count == 5){
356                 String[] str = line.split("\\s+");
357                 instance.customerNum = customerNum;  //顾客的数量
358                 instance.nodeNum = nodeNum;
359                 instance.vehicleNum = Integer.parseInt(str[1]);
360                 instance.vehicleCapacity = Integer.parseInt(str[2]);
361             }else if(count >= 10 && count <= 10 + customerNum ){
             //读取一部分数据
362                 //读取全部数据就只是count >= 10
363                 //读取部分数据就是count >= 10 && count <= 10 + customerNum
364                 String[] str = line.split("\\s+");
365                 Node node = new Node();
366                 node.ID = Integer.parseInt(str[1]);
367                 node.Xcoor = Integer.parseInt(str[2]);
368                 node.Ycoor = Integer.parseInt(str[3]);
369                 node.demand = Integer.parseInt(str[4]);
370                 node.readyTime = Integer.parseInt(str[5]);
371                 node.dueTime = Integer.parseInt(str[6]);
372                 node.serviceTime = Integer.parseInt(str[7]);
373                 Nodes.add(node);  //将node加入列表Nodes中
374             }
375         }
376         //按照Solomon的模型，将起始点复制一份加入Nodes列表当中
377         Node node1 = new Node();
```

```
            node1.ID = instance.customerNum + 1;
            node1.Xcoor = Nodes.get(0).Xcoor;
            node1.Ycoor = Nodes.get(0).Ycoor;
            node1.demand = Nodes.get(0).demand;
            node1.readyTime = Nodes.get(0).readyTime;
            node1.dueTime = Nodes.get(0).dueTime;
            node1.serviceTime = Nodes.get(0).serviceTime;
            Nodes.add(node1);

            instance.Nodes = Nodes;  //将Nodes的集合添加到instance中
            br.close();  //关闭资源

            return instance;
        }

        public static void printInstance(Instance instance){
            System.out.println("vehicleNum" + "\t\t: " + instance.vehicleNum);
            System.out.println("vehicleCapacity" + "\t\t: " + instance.
           vehicleCapacity);
            for(Node node : instance.Nodes){  //这里用了增强for
                System.out.print(node.ID + "\t");
                System.out.print(node.Xcoor + "\t");
                System.out.print(node.Ycoor + "\t");
                System.out.print(node.demand + "\t");
                System.out.print(node.readyTime + "\t");
                System.out.print(node.dueTime + "\t");
                System.out.print(node.serviceTime + "\n");
            }

        }

        //截断小数3.26434-->3.2
        public static double double_truncate(double v){
            int iv = (int) v;
            if(iv+1 - v <= gap)
                return iv+1;
            double dv = (v - iv) * 10;
            int idv = (int) dv;
            double rv = iv + idv / 10.0;
            return rv;
        }
    }
```

2. Instance 类

Instance 类代码如下：

Instance.java

```
package VRPTW_Hsinglu;

import java.io.BufferedReader;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class Instance {
    int customerNum;
    int nodeNum;
    int vehicleCapacity;
    int vehicleNum;
    List<Node> Nodes;

}
```

3. Node 类

Node 类代码如下：

Node.java

```
package VRPTW_Hsinglu;

public class Node {
    int ID;
    int Xcoor;
    int Ycoor;
    int demand;
    int readyTime;
    int dueTime;
    int serviceTime;
}
```

4. Result 类

Result 类代码如下：

Result.java

```
package VRPTW_Hsinglu;

public class Result {
    double objective;
    double[][][] X;
    double[][] S;
}
```

5. 算例格式

算例格式如下：

算例 1

```
C101

VEHICLE
NUMBER     CAPACITY
  25         200

CUSTOMER
CUST NO.  XCOORD.   YCOORD.    DEMAND   READY TIME  DUE DATE   SERVICE
       TIME

    0       40         50          0          0        1236          0
    1       45         68         10        912         967         90
    2       45         70         30        825         870         90
    3       42         66         10         65         146         90
    4       42         68         10        727         782         90
    5       42         65         10         15          67         90
    6       40         69         20        621         702         90
    7       40         66         20        170         225         90
    8       38         68         20        255         324         90
    9       38         70         10        534         605         90
   10       35         66         10        357         410         90
   11       35         69         10        448         505         90
   12       25         85         20        652         721         90
   13       22         75         30         30          92         90
   14       22         85         10        567         620         90
   15       20         80         40        384         429         90
   16       20         85         40        475         528         90
   17       18         75         20         99         148         90
   18       15         75         20        179         254         90
```

29	19	15	80	10	278	345	90
30	20	30	50	10	10	73	90
31	21	30	52	20	914	965	90
32	22	28	52	20	812	883	90
33	23	28	55	10	732	777	90
34	24	25	50	10	65	144	90
35	25	25	52	40	169	224	90
36	26	25	55	10	622	701	90
37	27	23	52	10	261	316	90
38	28	23	55	20	546	593	90
39	29	20	50	10	358	405	90
40	30	20	55	10	449	504	90
41	31	10	35	20	200	237	90
42	32	10	40	30	31	100	90
43	33	8	40	40	87	158	90
44	34	8	45	20	751	816	90
45	35	5	35	10	283	344	90
46	36	5	45	10	665	716	90
47	37	2	40	20	383	434	90
48	38	0	40	30	479	522	90
49	39	0	45	20	567	624	90
50	40	35	30	10	264	321	90
51	41	35	32	10	166	235	90
52	42	33	32	20	68	149	90
53	43	33	35	10	16	80	90
54	44	32	30	10	359	412	90
55	45	30	30	10	541	600	90
56	46	30	32	30	448	509	90
57	47	30	35	10	1054	1127	90
58	48	28	30	10	632	693	90
59	49	28	35	10	1001	1066	90
60	50	26	32	10	815	880	90
61	51	25	30	10	725	786	90
62	52	25	35	10	912	969	90
63	53	44	5	20	286	347	90
64	54	42	10	40	186	257	90
65	55	42	15	10	95	158	90
66	56	40	5	30	385	436	90
67	57	40	15	40	35	87	90
68	58	38	5	30	471	534	90
69	59	38	15	10	651	740	90
70	60	35	5	20	562	629	90

71	61	50	30	10	531	610	90
72	62	50	35	20	262	317	90
73	63	50	40	50	171	218	90
74	64	48	30	10	632	693	90
75	65	48	40	10	76	129	90
76	66	47	35	10	826	875	90
77	67	47	40	10	12	77	90
78	68	45	30	10	734	777	90
79	69	45	35	10	916	969	90
80	70	95	30	30	387	456	90
81	71	95	35	20	293	360	90
82	72	53	30	10	450	505	90
83	73	92	30	10	478	551	90
84	74	53	35	50	353	412	90
85	75	45	65	20	997	1068	90
86	76	90	35	10	203	260	90
87	77	88	30	10	574	643	90
88	78	88	35	20	109	170	90
89	79	87	30	10	668	731	90
90	80	85	25	10	769	820	90
91	81	85	35	30	47	124	90
92	82	75	55	20	369	420	90
93	83	72	55	10	265	338	90
94	84	70	58	20	458	523	90
95	85	68	60	30	555	612	90
96	86	66	55	10	173	238	90
97	87	65	55	20	85	144	90
98	88	65	60	30	645	708	90
99	89	63	58	10	737	802	90
100	90	60	55	10	20	84	90
101	91	60	60	10	836	889	90
102	92	67	85	20	368	441	90
103	93	65	85	40	475	518	90
104	94	65	82	10	285	336	90
105	95	62	80	30	196	239	90
106	96	60	80	10	95	156	90
107	97	60	85	30	561	622	90
108	98	58	75	20	30	84	90
109	99	55	80	10	743	820	90
110	100	55	85	20	647	726	90

7.1.3 Java 调用 Gurobi 求解 VRPTW

下面代码中类的功能和 Java 调用 CPLEX 的代码基本相同。

1. Main 类

Main 类代码如下：

Main.java

```
package VRPTW_Hsinglu;

/**
 * @author: Hsinglu Liu
 * @School: Tsinghua University
 * @操作说明：修改读取文件的文件名，求解C101和R101两个算例，
 *          修改customerNum来截取100个客户点中前customerNum个客户点，求解不同规
 *          模的VRPTW
 * 算例来源：Solomon于1987年发表在OR上的论文
 *
 */
import gurobi.*;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class Main {

    //算例截取参数
    //customerNum = 30, 表示截取 C101 中 100 个顾客点的前 30 个顾客点作为测试数据
    static final int customerNum = 100;
    static final int nodeNum = customerNum + 2;     //节点的数量=客户点的数量
        //+2
    static final int vehicleNum = 25;
    static final int big_M = 10000;
    public static void main(String[] args) throws IOException, GRBException
       {
        //首先写入算例的路径
        String path = "E:\\MyCode\\JavaCode\\JavaCallGurobi\\src\\r101.txt";

        //根据算例路径，读取算例数据
        Instance instance = readData(path);
```

```
33
34          //打印出算例数据信息
35          printInstance(instance);
36
37
38          //调用Gurobi构建VRPTW模型
39          GRBEnv env = new GRBEnv("mip1.log");
40          GRBModel model = new GRBModel(env);
41
42          //创建变量
43          GRBVar X[][][] = new GRBVar[instance.nodeNum][instance.nodeNum]
             [instance.vehicleNum];
44          GRBVar S[][] = new GRBVar[instance.nodeNum][instance.vehicleNum];
45
46          //设置变量类型  instance.Nodes.get(i).dueTime
47          for(int i = 0; i < instance.nodeNum; i++) {
48              for(int k = 0; k < instance.vehicleNum; k++) {
49                  S[i][k] = model.addVar(instance.Nodes.get(i).readyTime,
        instance.Nodes.get(i).dueTime, 0.0, GRB.CONTINUOUS, "S_"+i+"_"+k);
50                  //System.out.println(instance.Nodes.get(i).readyTime + "---" +
        //instance.Nodes.get(i).dueTime);
51                  for(int j = 0; j < instance.nodeNum; j++) {
52                      X[i][j][k] = model.addVar(0, 1, 0.0, GRB.BINARY, "X_" +
        i + "_" + j + "_" + k);
53                  }
54              }
55          }
56
57          //建立目标函数
58          GRBLinExpr obj = new GRBLinExpr();
59          for(int i = 0; i < instance.nodeNum; i++) {
60              for(int j = 0; j < instance.nodeNum; j++) {
61                  for(int k = 0; k < instance.vehicleNum; k++) {
62                      if(i != j) {
63                          obj.addTerm(instance.disMatrix[i][j], X[i][j][k]);
64                      }
65                  }
66              }
67          }
68          model.setObjective(obj, GRB.MINIMIZE);
69
70          //约束(7.2)
```

```
71          for(int i = 1; i < instance.nodeNum - 1; i++) {
72              GRBLinExpr expr = new GRBLinExpr();
73              for(int j = 0; j < instance.nodeNum; j++) {
74                  if(i != j) {
75                      for(int k = 0; k < instance.vehicleNum; k++) {
76                          expr.addTerm(1.0, X[i][j][k]);
77                      }
78                  }
79              }
80              model.addConstr(expr, GRB.EQUAL, 1, "c2");
81          }
82
83          //约束（7.3）
84          for(int k = 0; k < instance.vehicleNum; k++) {
85              GRBLinExpr expr = new GRBLinExpr();
86              for(int i = 1; i < instance.nodeNum - 1; i++) {
87                  for(int j = 0; j < instance.nodeNum; j++) {
88                      if(i != j) {
89                          expr.addTerm(instance.Nodes.get(i).demand, X[i][j][k
    ]);
90                      }
91                  }
92              }
93              model.addConstr(expr, GRB.LESS_EQUAL, instance.vehicleCapacity,
    "c3");
94          }
95
96          //约束（7.4）
97          for(int k = 0; k < instance.vehicleNum; k++) {
98              GRBLinExpr expr = new GRBLinExpr();
99              for(int j = 1; j < instance.nodeNum; j++) {
100                 expr.addTerm(1, X[0][j][k]);
101             }
102             model.addConstr(expr, GRB.EQUAL, 1, "c4");
103         }
104
105         //约束（7.5）
106         for(int h = 1; h < instance.nodeNum - 1; h++) {
107             for(int k = 0; k < instance.vehicleNum; k++) {
108                 GRBLinExpr expr = new GRBLinExpr();
109                 for(int i = 0; i < instance.nodeNum; i++) {
110                     if(i != h) {
```

```
                        expr.addTerm(1, X[i][h][k]);
                    }
                }
                for(int j = 0; j < instance.nodeNum; j++) {
                    if(h != j) {
                        expr.addTerm(-1, X[h][j][k]);
                    }
                }
                model.addConstr(expr, GRB.EQUAL, 0, "c5");
            }
        }

        //约束 (7.6)
        for(int k = 0; k < instance.vehicleNum; k++) {
            GRBLinExpr expr = new GRBLinExpr();
            for(int i = 0; i < instance.nodeNum - 1; i++) {
                expr.addTerm(1, X[i][instance.nodeNum - 1][k]);
            }
            model.addConstr(expr, GRB.EQUAL, 1, "c6");
        }

        //约束 (7.7)
        for(int i = 0; i < instance.nodeNum; i++) {
            for(int j = 0; j < instance.nodeNum; j++) {
                if(i != j) {
                    for(int k = 0; k < instance.vehicleNum; k++) {
                        GRBLinExpr expr = new GRBLinExpr();
                        // S_ik - S_jk + M * X_ijk <= M - T_ij
                        expr.addTerm(1, S[i][k]);
                        expr.addTerm(-1, S[j][k]);
                        expr.addTerm(big_M, X[i][j][k]);
                        model.addConstr(expr, GRB.LESS_EQUAL, big_M -
    instance.disMatrix[i][j], "c7");
                    }
                }
            }
        }

        //约束
        for(int i = 0; i < instance.nodeNum; i++) {
            for(int k = 0; k < instance.vehicleNum; k++) {
                model.addConstr(X[i][i][k], GRB.EQUAL, 0, "c78");
```

```
            }
        }

        //求解模型
        model.optimize();
        //得到解
        ArrayList<ArrayList<Integer>> routes = new ArrayList<ArrayList<
    Integer>>();
        for(int k = 0; k < instance.vehicleNum; k++) {
            System.out.println();
            ArrayList<Integer> tour = new ArrayList<Integer>();
            int j = 0;
            for(int i = 0; i < instance.nodeNum; i++) {
                i = j;
                for(j = 0; j < instance.nodeNum; j++) {
                    if(X[i][j][k].get(GRB.DoubleAttr.X) > 0) {
                        System.out.println(i + "-" + j);
                        tour.add(i);
                        break;
                    }
                }
            }
            System.out.print("0");
            tour.add(0);
            routes.add(tour);
        }
        for(int i = 0; i < routes.size(); i++) {
            if(routes.get(i).size() <= 2) {
                routes.remove(i);
                i = i - 1;
            }
        }

        System.out.println("\n\n----以下是路径------");
        for(int i = 0; i < routes.size(); i++) {
            for(int j = 0; j < routes.get(i).size(); j++) {
                if(j != routes.get(i).size() - 1) {
                    System.out.print(routes.get(i).get(j) + "-");
                }else {
                    System.out.print(routes.get(i).get(j));
                }

```

```
            }
            System.out.println();
        }


        model.dispose();
        env.dispose();

    }

    //读取算例数据的方法
    public static Instance readData(String path) throws IOException {
        //首先创建存放所有节点的数据的列表

        Instance instance = new Instance();
        List<Node> Nodes = new ArrayList<Node>();

        //创建文件读入对象
        BufferedReader br = new BufferedReader(new FileReader(path));
        //这里需要抛异常

        //进行读取操作
        String line = null;
        int count = 0;
        while((line = br.readLine()) != null){
            count += 1;
            System.out.println(line + count);
            if(count == 5){
                String[] str = line.split("\\s+");
                instance.customerNum = customerNum;  //顾客的数量
                instance.nodeNum = nodeNum;
                instance.vehicleNum = Integer.parseInt(str[1]);
                instance.vehicleCapacity = Integer.parseInt(str[2]);
            }else if(count >= 10 && count <= 10 + customerNum ){
        //读取一部分数据
                //读取全部数据就只是count >= 10
                //读取部分数据就是count >= 10 && count <= 10 + customerNum
                String[] str = line.split("\\s+");
                Node node = new Node();
                node.ID = Integer.parseInt(str[1]);
                node.Xcoor = Integer.parseInt(str[2]);
                node.Ycoor = Integer.parseInt(str[3]);
```

```
                node.demand = Integer.parseInt(str[4]);
                node.readyTime = Integer.parseInt(str[5]);
                node.dueTime = Integer.parseInt(str[6]);
                node.serviceTime = Integer.parseInt(str[7]);
                Nodes.add(node);  //将node加入列表Nodes中
            }
        }
        //按照Solomon的模型，将起始点复制一份加入Nodes列表当中
        Node node1 = new Node();
        node1.ID = instance.customerNum + 1;
        node1.Xcoor = Nodes.get(0).Xcoor;
        node1.Ycoor = Nodes.get(0).Ycoor;
        node1.demand = Nodes.get(0).demand;
        node1.readyTime = Nodes.get(0).readyTime;
        node1.dueTime = Nodes.get(0).dueTime;
        node1.serviceTime = Nodes.get(0).serviceTime;
        Nodes.add(node1);

        instance.Nodes = Nodes;  //将Nodes的集合添加到instance中
        br.close();  //关闭资源

        //下面计算距离矩阵
        instance.disMatrix = new double[instance.nodeNum][instance.nodeNum];
        for(int i = 0; i < instance.nodeNum; i++) {
            for(int j = 0; j < instance.nodeNum; j++) {
                double x1 = instance.Nodes.get(i).Xcoor;
                double y1 = instance.Nodes.get(i).Ycoor;
                double x2 = instance.Nodes.get(j).Xcoor;
                double y2 = instance.Nodes.get(j).Ycoor;
                instance.disMatrix[i][j] = Math.sqrt((x1 - x2)*(x1 - x2) +
     (y1 - y2)*(y1 - y2));
            }
        }

        return instance;
    }

    public static void printInstance(Instance instance){
        System.out.println("vehicleNum" + "\t\t: " + instance.vehicleNum);
        System.out.println("vehicleCapacity" + "\t\t: " + instance.
      vehicleCapacity);
        for(Node node : instance.Nodes){  //这里用了增强for
```

```
            System.out.print(node.ID + "\t");
            System.out.print(node.Xcoor + "\t");
            System.out.print(node.Ycoor + "\t");
            System.out.print(node.demand + "\t");
            System.out.print(node.readyTime + "\t");
            System.out.print(node.dueTime + "\t");
            System.out.print(node.serviceTime + "\n");
        }

    }
}
```

2. Node 类

Node 类代码如下：

Node.java

```
package VRPTW_Hsinglu;

public class Node {
    int ID;
    int Xcoor;
    int Ycoor;
    int demand;
    int readyTime;
    int dueTime;
    int serviceTime;

}
```

3. Instance 类

Instance 类代码如下：

Instance.java

```
package VRPTW_Hsinglu;

import java.io.BufferedReader;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
```

```
9
10 public class Instance {
11     int customerNum;
12     int nodeNum;
13     int vehicleCapacity;
14     int vehicleNum;
15     List<Node> Nodes;
16     double[][] disMatrix;
17 }
```

4. 算例格式

算例格式同 7.1.2 节，均为 Solomon 的 VRP 标杆算例。

7.1.4 Python 调用 Gurobi 求解 VRPTW

1. Main.py

Main.py 代码如下：

Main.py

```
1  # _*_coding:utf-8 _*_
2  from __future__ import print_function
3  from gurobipy import *
4  import re
5  import math
6  import matplotlib.pyplot as plt
7  import numpy
8  import pandas as pd
9
10 class Data:
11     customerNum = 0
12     nodeNum     = 0
13     vehicleNum  = 0
14     capacity    = 0
15     cor_X       = []
16     cor_Y       = []
17     demand      = []
18     serviceTime = []
19     readyTime   = []
20     dueTime     = []
21     disMatrix   = [[]]   # 读取数据
22
23 class Solution:
```

```
    ObjVal = 0
    X = [[[]]]
    S = [[]]
    routes = [[]]
    routeNum = 0

    def __init__(self):
        self.ObjVal = model.ObjVal
        # X_ijk
        self.X = [[([0] * data.vehicleNum) for i in range(data.nodeNum)]
       for j in range(data.nodeNum)]
        # S_ik
        self.S = [([0] * data.nodeNum) for j in range(data.vehicleNum)]
        # routes
        self.routes = [[]]

    def getSolution(self, data, model):
        solution = Solution()
        solution.ObjVal = model.ObjVal
        # X_ijk
        solution.X = [[([0] * data.vehicleNum) for i in range(data.nodeNum)]
         for j in range(data.nodeNum)]
        # S_ik
        solution.S = [([0] * data.nodeNum) for j in range(data.vehicleNum)]
        # routes
        solution.routes = [[]]

        # get the solutions of decision variables
        print("\n\n-------------这个是最优解---------------")
        for m in model.getVars():
            str = re.split(r"_", m.VarName)
            if(str[0] == "X" and m.x == 1)
                print("str[1] = %d" % int(str[1]))
                print("str[2] = %d" % int(str[2]))
                print("str[3] = %d" % int(str[3]))
                print("m.x = ", end = "" )
                print(m.x)
                solution.X[int(str[1])][int(str[2])][int(str[3])] = m.x
                print(str, end = "")
                print(" = %d" % m.x)
            elif(str[0] == "S" and m.x == 1):
                solution.S[int(str[1])][int(str[2])] = m.x
```

```
64
65          print("-----solution = ------")
66          for k in range(data.vehicleNum):
67              for i in range(data.nodeNum):
68                  for j in range(data.nodeNum):
69                      if(solution.X[i][j][k] > 0 and (not(i == 0 and j == data
    .nodeNum - 1))):
70                          print("x[{0},{1},{2}] = {3}" .format(i, j, k,
    solution.X[i][j][k]))
71
72          print(solution.X)
73          # get the route of vehicles from the value of decision variables
74          for k in range(data.vehicleNum):
75              i = 0
76              subRoute = []
77              subRoute.append(i)
78              finish = False
79              while(not finish):
80                  for j in range(data.nodeNum):
81                      if(solution.X[i][j][k] > 0):
82                          subRoute.append(j)
83                          i = j
84                          if(j == data.nodeNum - 1):
85                              finish = True
86
87              if(len(subRoute) >= 3):
88                  subRoute[len(subRoute) - 1] = 0
89                  solution.routes.append(subRoute)
90                  solution.routeNum = solution.routeNum + 1
91
92          print("\n\n ------Route of Vehicles ------- ")
93          print(solution.routes)
94
95          print("\n\n ------Drawing the Graph ------- ")
96          # draw the route graph
97          # draw all the nodes first
98          # data1 = Data()
99          # readData(data1, path, 100)
100         fig = plt.figure(0)
101         plt.xlabel('x')
102         plt.ylabel('y')
103         plt.title('All Customers')
```

```
        '''
        marker='o'
        marker=','
        marker='.'
        marker=(9, 3, 30)
        marker='+'
        marker='v'
        marker='^'
        marker='<'
        marker='>'
        marker='1'
        marker='2'
        marker='3'
        'magenta'
        red         blue         green
        '''
        plt.scatter(data.cor_X[0], data.cor_Y[0], c='blue', alpha=1, marker=
    ',', linewidths=3, label='depot')
        plt.scatter(data.cor_X[1:-1], data.cor_Y[1:-1], c='black', alpha=1,
    marker='o', linewidths=3, label='customer')
        # c='red'定义为红色，alpha是透明度，marker是绘制的样式

        # draw the route
        for k in range(solution.routeNum + 1):   #注意这里是有问题的
            for i in range(len(solution.routes[k]) - 1):
                a = solution.routes[k][i]
                b = solution.routes[k][i+1]
                x = [data.cor_X[a], data.cor_X[b]]
                y = [data.cor_Y[a], data.cor_Y[b]]
                plt.plot(x, y, 'k', linewidth = 1)  # r--

        # plt.grid(True)
        plt.grid(False)
        plt.legend(loc='best')
        plt.show()

        # print(solution.route_UAV)

        return solution

# function to read data from .txt files
def readData(data, path, customerNum):
```

```
    data.customerNum = customerNum
    data.nodeNum = customerNum + 2
    f = open(path, 'r')
    lines = f.readlines()
    count = 0
    # read the info
    for line in lines:
        count = count + 1
        if(count == 3):
            line = line[:-1]
            str = re.split(r" +", line)
            data.vehicleNum = int(str[0])
            data.capacity = float(str[1])
        elif(count >= 7 and count <= 7 + customerNum):
            line = line[:-1]
            str = re.split(r" +", line)
            data.cor_X.append(float(str[2]))
            data.cor_Y.append(float(str[3]))
            data.demand.append(float(str[4]))
            data.readyTime.append(float(str[5]))
            data.dueTime.append(float(str[6]))
            data.serviceTime.append(float(str[7]))

    data.cor_X.append(data.cor_X[0])
    data.cor_Y.append(data.cor_Y[0])
    data.demand.append(data.demand[0])
    data.readyTime.append(data.readyTime[0])
    data.dueTime.append(data.dueTime[0])
    data.serviceTime.append(data.serviceTime[0])

    # compute the distance matrix
    data.disMatrix = [([0] * data.nodeNum) for p in range(data.nodeNum)]
       # 初始化距离矩阵的维度,防止浅拷贝
    # data.disMatrix = [[0] * nodeNum] * nodeNum] 这个是浅拷贝，容易重复
    for i in range(0, data.nodeNum):
        for j in range(0, data.nodeNum):
            temp = (data.cor_X[i] - data.cor_X[j])**2 + (data.cor_Y[i] -
       data.cor_Y[j])**2
            data.disMatrix[i][j] = math.sqrt(temp)
            # print("%6.2f" % (math.sqrt(temp)), end = " ")
            temp = 0

```

```
    return data

def printData(data, customerNum):
    print("下面打印数据\n")
    print("vehicle number = %4d" % data.vehicleNum)
    print("vehicle capacity = %4d" % data.capacity)
    for i in range(len(data.demand)):
        print('{0}\t{1}\t{2}\t{3}'.format(data.demand[i], data.readyTime[i],
       data.dueTime[i],  data.serviceTime[i]))

    print("-------距离矩阵-------\n")
    for i in range(data.nodeNum):
        for j in range(data.nodeNum):
            #print("%d   %d" % (i, j))
            print("%6.2f" % (data.disMatrix[i][j]), end = " ")
        print()

# reading data
data = Data()

path = 'C:\Users\hsingluLiu\eclipse-workspace\PythonCallGurobi_Applications\
      VRPTW\R101.txt'
customerNum = 100
readData(data, path, customerNum)
printData(data, customerNum)

# =========build the model===========
big_M = 10000
# construct the model object
model = Model("VRPYW")

# Initialize variables
# create variables: Muiti-dimension vector: from inner to outer
# X_ijk
X = [[[[] for k in range(data.vehicleNum)] for j in range(data.nodeNum)] for
       i in range(data.nodeNum)]

# S_ik
S = [[[] for k in range(data.vehicleNum)] for i in range(data.nodeNum)]

for i in range(data.nodeNum):
    for k in range(data.vehicleNum):
```

```
222         name1 = 's_' + str(i) +'_' + str(k)
223         S[i][k] = model.addVar(data.readyTime[i], data.dueTime[i], vtype =
        GRB.CONTINUOUS, name = name1)
224         for j in range(data.nodeNum):
225               #if(i != j):
226                 name2 = 'X_' + str(i) + "_" + str(j) + "_"  + str(k)
227                 X[i][j][k] = model.addVar(0, 1, vtype = GRB.BINARY, name =
        name2)
228
229 # Add constraints
230 # create the objective expression
231 obj = LinExpr(0)
232 for i in range(data.nodeNum):
233     for j in range(data.nodeNum):
234         if(i != j):
235             for k in range(data.vehicleNum):
236                 obj.addTerms(data.disMatrix[i][j], X[i][j][k])
237 #print(model.getObjective()) #这个可以打印出目标函数
238
239 # add the objective function into the model
240 model.setObjective(obj, GRB.MINIMIZE)
241
242 # constraint (1)
243 for i in range(1, data.nodeNum - 1): # 这里需要注意i的取值范围，否则可能会加
          # 入空约束
244     expr = LinExpr(0)
245     for j in range(data.nodeNum):
246         if(i != j):
247             for k in range(data.vehicleNum):
248                 if(i != 0 and i != data.nodeNum - 1):
249                     expr.addTerms(1, X[i][j][k])
250
251     model.addConstr(expr == 1, "c1")
252     expr.clear()
253
254 # constraint (2)
255 for k in range(data.vehicleNum):
256     expr = LinExpr(0)
257     for i in range(1, data.nodeNum - 1):
258         for j in range(data.nodeNum):
259             if(i != 0 and i != data.nodeNum - 1 and i != j):
260                 expr.addTerms(data.demand[i], X[i][j][k])
```

```
261     model.addConstr(expr <= data.capacity, "c2")
262     expr.clear()
263
264 # constraint (3)
265 for k in range(data.vehicleNum):
266     expr = LinExpr(0)
267     for j in range(1, data.nodeNum): # 此处注意，不能有i == j的情况出现
268         expr.addTerms(1.0, X[0][j][k])
269     model.addConstr(expr == 1.0, "c3")
270     expr.clear()
271
272 # constraint (4)
273 for k in range(data.vehicleNum):
274     for h in range(1, data.nodeNum - 1):
275         expr1 = LinExpr(0)
276         expr2 = LinExpr(0)
277         for i in range(data.nodeNum):
278             if(h != i):
279                 expr1.addTerms(1, X[i][h][k])
280
281         for j in range(data.nodeNum):
282             if(h != j):
283                 expr2.addTerms(1, X[h][j][k])
284
285         model.addConstr(expr1 == expr2, "c4")
286         expr1.clear()
287         expr2.clear()
288
289 # constraint (5)
290 for k in range(data.vehicleNum):
291     expr = LinExpr(0)
292     for i in range(data.nodeNum - 1): #这个地方也要注意，是data.nodeNum - 1,
        # 不是data.nodeNum
293         expr.addTerms(1, X[i][data.nodeNum - 1][k])
294     model.addConstr(expr == 1, "c5")
295     expr.clear()
296
297 # constraint (6)
298 for k in range(data.vehicleNum):
299     for i in range(data.nodeNum):
300         for j in range(data.nodeNum):
301             if(i != j):
```

```
                model.addConstr(S[i][k] + data.disMatrix[i][j] - S[j][k] <=
        big_M  - big_M * X[i][j][k], "c6")

# solve the problem
# model.setAttr("ub", model.getVarByName('X_0_2_3'), 0)
# model.setAttr("modelSense", -1)

model.write('a.lp')
model.optimize()
print("\n\n-----optimal value-----")
print(model.ObjVal)

for m in model.getVars():
    if(m.x == 1):
        print("%s \t %d" % (m.varName, m.x))

# fig = plt.figure(0)
# plt.xlabel('x')
# plt.ylabel('y')
# plt.title('All Customers')
# plt.scatter(data.cor_X[0], data.cor_Y[0], c='red', alpha=1, marker=',',
      # linewidths=10, label='depot')
# plt.text(data.cor_X[0]+1, data.cor_Y[0]+1, 'Depot', color = 'r', fontsize
      # =30)
# plt.scatter(data.cor_X[1:-1], data.cor_Y[1:-1], c='black', alpha=1, marker
       # ='o', linewidths=3, label='customer')
       # c='red'定义为红色，alpha是透明度，marker是绘制的样式
# plt.text(data.cor_X[34]-6, data.cor_Y[34]+4, 'Customers', color = 'r',
      # fontsize=30)

# # plt.grid(True)
# plt.grid(False)
# plt.legend(loc='best')
# plt.show()

# get the solution info
# solution = Solution()
# solution = solution.getSolution(data, model)
```

2. 结果展示

Solomon 算例 C101 的最优解路径图如图 7.1 所示。

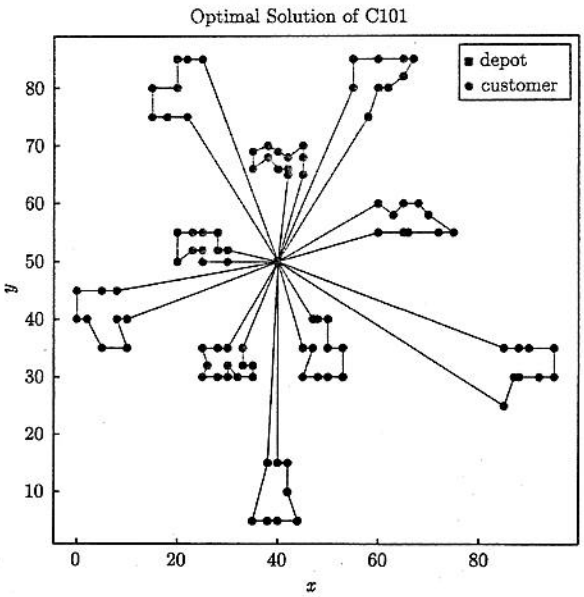


图 7.1　C101 最优解路径图

C101 的最优解如下：

C101 的最优解

```
Problem Set: C101
Route: 1, 13-17-18-19-15-16-14-12-0
Route: 2, 43-42-41-40-44-46-45-48-51-50-52-49-47-0
Route: 3, 90-87-86-83-82-84-85-88-89-91-0
Route: 4, 67-65-63-62-74-72-61-64-68-66-69-0
Route: 5, 98-96-95-94-92-93-97-100-99-0
Route: 6, 5-3-7-8-10-11-9-6-4-2-1-75-0
Route: 7, 20-24-25-27-29-30-28-26-23-22-21-0
Route: 8, 81-78-76-71-70-73-77-79-80-0
Route: 9, 57-55-54-53-56-58-60-59-0
Route: 10, 32-33-31-35-37-38-39-36-34-0
Vehicle capacity: 200.00
Routes: 10
Total travel distance: 828.93664000
Route: 1, length: 8, distance:       95.88470000, max load:  190.00
Route: 2, length: 13, distance:       64.80744000, max load:  160.00
Route: 3, length: 10, distance:       76.06953000, max load:  170.00
Route: 4, length: 11, distance:       59.40309000, max load:  200.00
Route: 5, length: 9, distance:       95.94311000, max load:  190.00
Route: 6, length: 12, distance:       59.61803000, max load:  180.00
```

```
Route: 7, lengthen: 11, distance:       50.80358000, max load:  170.00
Route: 8, length: 9, distance:      127.29747000, max load:  150.00
Route: 9, length: 8, distance:      101.88255000, max load:  200.00
Route: 10, length: 9, distance:      97.22714000, max load:  200.00
```

Solomon 算例 R101 的最优解路径图如图 7.2 所示。

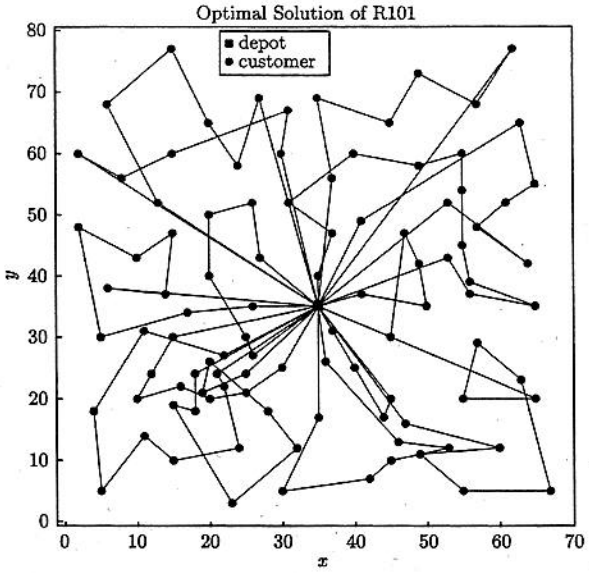


图 7.2 R101 最优解路径图

R101 的最优解如下：

R101 的最优解

```
Problem Set: R101
Route: 1, 92-42-15-87-57-97-0
Route: 2, 27-69-76-79-3-54-24-80-0
Route: 3, 95-98-16-86-91-100-0
Route: 4, 36-47-19-8-46-17-0
Route: 5, 14-44-38-43-13-0
Route: 6, 39-23-67-55-25-0
Route: 7, 31-88-7-10-0
Route: 8, 62-11-90-20-32-70-0
Route: 9, 2-21-73-41-56-4-0
Route: 10, 5-83-61-85-37-93-0
Route: 11, 52-6-0
Route: 12, 65-71-81-50-68-0
```

```
Route: 13, 59-99-94-96-0
Route: 14, 72-75-22-74-58-0
Route: 15, 33-29-78-34-35-77-0
Route: 16, 30-51-9-66-1-0
Route: 17, 45-82-18-84-60-89-0
Route: 18, 28-12-40-53-26-0
Route: 19, 63-64-49-48-0
Vehicle capacity: 200.00
Routes: 19
Total travel distance: 1650.79864000
Route: 1, len: 6, distance:       86.77278000, max load:   60.00
Route: 2, len: 8, distance:       95.32472000, max load:   98.00
Route: 3, len: 6, distance:       75.20345000, max load:  102.00
Route: 4, len: 6, distance:      132.48994000, max load:   61.00
Route: 5, len: 5, distance:      100.86701000, max load:   84.00
Route: 6, len: 5, distance:      110.15853000, max load:   93.00
Route: 7, len: 4, distance:       68.42601000, max load:   57.00
Route: 8, len: 6, distance:      103.89808000, max load:   71.00
Route: 9, len: 6, distance:       87.12985000, max load:   57.00
Route: 10, len: 6, distance:       74.50436000, max load:  121.00
Route: 11, len: 2, distance:       35.64697000, max load:   12.00
Route: 12, len: 5, distance:      118.41699000, max load:  110.00
Route: 13, len: 4, distance:       45.38764000, max load:   75.00
Route: 14, len: 5, distance:       60.59932000, max load:   87.00
Route: 15, len: 6, distance:      109.12702000, max load:   59.00
Route: 16, len: 5, distance:       95.88676000, max load:   82.00
Route: 17, len: 6, distance:       87.97649000, max load:   69.00
Route: 18, len: 5, distance:       55.63700000, max load:   75.00
Route: 19, len: 4, distance:      107.34572000, max load:   85.00
```

7.2 Python 调用 Gurobi 求解 TSP

首先，我们给出读取数据、打印数据、获得最终路径的函数。

- `readData(path, nodeNum)`：读取 txt 文件中的算例数据；
- `reportMIP(model, Routes)`：获得并打印最优解信息；
- `getValue(var_dict, nodeNum)`：获得决策变量的值，并存储到 np.array() 数组中；
- `getRoute(x_value)`：根据解 x_value 得到该解对应的路径。

functions.py

```
# _*_coding:utf-8 _*_
'''
@author: Hsinglu Liu
@version: 1.0
@Date: 2019.5.5
'''

from __future__ import print_function
from __future__ import division, print_function
from gurobipy import *
import re;
import math;
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import copy
from matplotlib.lines import lineStyles
import time

starttime = time.time()

# function to read data from .txt files
def readData(path, nodeNum):
    nodeNum = nodeNum;
    cor_X = []
    cor_Y = []

    f = open(path, 'r');
    lines = f.readlines();
    count = 0;
    # read the info
    for line in lines:
        count = count + 1;
        if(count >= 10 and count <= 10 + nodeNum):
            line = line[:-1]
            str = re.split(r" +", line)
            cor_X.append(float(str[2]))
            cor_Y.append(float(str[3]))

    # compute the distance matrix
    disMatrix = [([0] * nodeNum) for p in range(nodeNum)]; # 初始化距离矩阵
```

```
        # 的维度,防止浅拷贝
    # data.disMatrix = [[0] * nodeNum] * nodeNum]; 这个是浅拷贝，容易重复
    for i in range(0, nodeNum):
        for j in range(0, nodeNum):
            temp = (cor_X[i] - cor_X[j])**2 + (cor_Y[i] - cor_Y[j])**2;
            disMatrix[i][j] = (int)(math.sqrt(temp));
            temp = 0;

    return disMatrix;

def printData(disMatrix):
    print("-------cost matrix-------\n");
    for i in range(len(disMatrix)):
        for j in range(len(disMatrix)):
            #print("%d   %d" % (i, j));
            print("%6.1f" % (disMatrix[i][j]), end = " ");
             #print(disMatrix[i][j], end = " ");
           print();

def reportMIP(model, Routes):
    if model.status == GRB.OPTIMAL:
        print("Best MIP Solution: ", model.objVal, "\n")
        var = model.getVars()
        for i in range(model.numVars):
            if(var[i].x > 0):
                print(var[i].varName, " = ", var[i].x)
                print("Optimal route:", Routes[i])

def getValue(var_dict, nodeNum):
    x_value = np.zeros([nodeNum + 1, nodeNum + 1])
    for key in var_dict.keys():
        a = key[0]
        b = key[1]
        x_value[a][b] = var_dict[key].x

    return x_value

def getRoute(x_value):
 '''
 input: x_value的矩阵
 output:一条路径，[0, 4, 3, 7, 1, 2, 5, 8, 9, 6, 0]，像这样
 '''
```

```
    # 假如是5个点的算例，路径会是1-4-2-3-5-6这样的，因为加入了一个虚拟点
    # 也就是当路径长度为6时，停止，这个长度和x_value的长度相同
    x = copy.deepcopy(x_value)
    previousPoint = 0
    route_temp = [previousPoint]
    count = 0
    while(len(route_temp) < len(x)):
        #print('previousPoint: ', previousPoint )
        if(x[previousPoint][count] > 0):
            previousPoint = count
            route_temp.append(previousPoint)
            count = 0
            continue
        else:
            count += 1
    route_temp.append(0)
    return route_temp
```

7.2.1 TSP 的 MTZ 建模及调用 Gurobi 求解

第 2 章已经详细介绍了 TSP 的 MTZ 建模。这里直接给出其模型。

$$\min \quad \sum_i \sum_j c_{ij} x_{ij} \tag{7.10}$$

$$\text{s.t.} \quad \sum_{i \in V} x_{ij} = 1, \qquad \forall j \in \{2,3,\cdots N+1\}, i \neq j \tag{7.11}$$

$$\sum_{j \in V} x_{ij} = 1, \qquad \forall i \in \{1,2,\cdots N\}, i \neq j \tag{7.12}$$

$$\mu_i - \mu_j + N x_{ij} \leqslant N-1, \qquad \forall i \in \{1,2,\cdots N\}, j \in \{2,3,\cdots N+1\}, i \neq j \tag{7.13}$$

$$x_{ij} \in \{0,1\}, \mu_i \geqslant 0, \mu_i \in \mathbb{R}, \qquad \forall i \in V \tag{7.14}$$

其中每个约束的含义见第 2 章。

下面给出 Python 调用 Gurobi 求解上述模型的代码。测试算例仍然为 Solomon 的 VRP 标杆算例。

TSP MTZ.py

```
# nodeNum = 5
nodeNum = 10
path = 'solomon-100/in/c101.txt';
cost = readData(path, nodeNum)
printData(cost)

```

```
model = Model('TSP')

# creat decision variables
X = {}
mu = {}
for i in range(nodeNum + 1):
    mu[i] = model.addVar(lb = 0.0
                         , ub = 100 #GRB.INFINITY
                          # , obj = distance_initial
                         , vtype = GRB.CONTINUOUS
                         , name = "mu_" + str(i)
                        )

    for j in range(nodeNum + 1):
        if(i != j):
            X[i, j] = model.addVar(vtype = GRB.BINARY
                                   , name = 'x_' + str(i) + '_' + str(j)
                                   )

# set objective function
obj = LinExpr(0)
for key in X.keys():
    i = key[0]
    j = key[1]
    if(i < nodeNum and j < nodeNum):
        obj.addTerms(cost[key[0]][key[1]], X[key])
    elif(i == nodeNum):
        obj.addTerms(cost[0][key[1]], X[key])
    elif(j == nodeNum):
        obj.addTerms(cost[key[0]][0], X[key])

model.setObjective(obj, GRB.MINIMIZE)

# add constraints 1
for j in range(1, nodeNum + 1):
    lhs = LinExpr(0)
    for i in range(0, nodeNum):
        if(i != j):
            lhs.addTerms(1, X[i, j])
    model.addConstr(lhs == 1, name = 'visit_' + str(j))

# add constraints 2
```

```
for i in range(0, nodeNum):
    lhs = LinExpr(0)
    for j in range(1, nodeNum + 1):
        if(i != j):
            lhs.addTerms(1, X[i, j])
    model.addConstr(lhs == 1, name = 'visit_' + str(j))

# add MTZ constraints
for i in range(0, nodeNum):
    for j in range(1, nodeNum + 1):
        if(i != j):
            model.addConstr(mu[i] - mu[j] + 100 * X[i, j] <= 100 - 1)

model.write('model.lp')
model.optimize()

print('Obj:', model.ObjVal)
x_value = getValue(X, nodeNum)
route = getRoute(x_value)
print('optimal route:', route)
```

我们取前 10 个点来测试，运行结果如下：

TSP MTZ.py

```
Explored 129556 nodes (574360 simplex iterations) in 2.25 seconds
Optimal solution found (tolerance 1.00e-04)
Best objective 4.800000000000e+01, best bound 4.800000000000e+01, gap
      0.0000%
Obj: 48.0
optimal route: [0, 7, 8, 9, 6, 4, 2, 1, 3, 5, 10, 0]
```

7.2.2 TSP：Python 调用 Gurobi 实现 callback 添加消除子环路约束

我们已经在第 2 章对该模型做了详细介绍，这里直接给出完整模型。

$$\min \quad \sum_i \sum_j c_{ij} x_{ij} \tag{7.15}$$

$$\text{s.t.} \quad \sum_{i \in V} x_{ij} = 1, \qquad \forall j \in V, i \neq j \tag{7.16}$$

$$\sum_{j \in V} x_{ij} = 1, \qquad \forall i \in V, i \neq j \tag{7.17}$$

$$\sum_{i,j \in S} x_{ij} \leqslant |S| - 1, \qquad 2 \leqslant |S| \leqslant N - 1, S \subseteq V \tag{7.18}$$